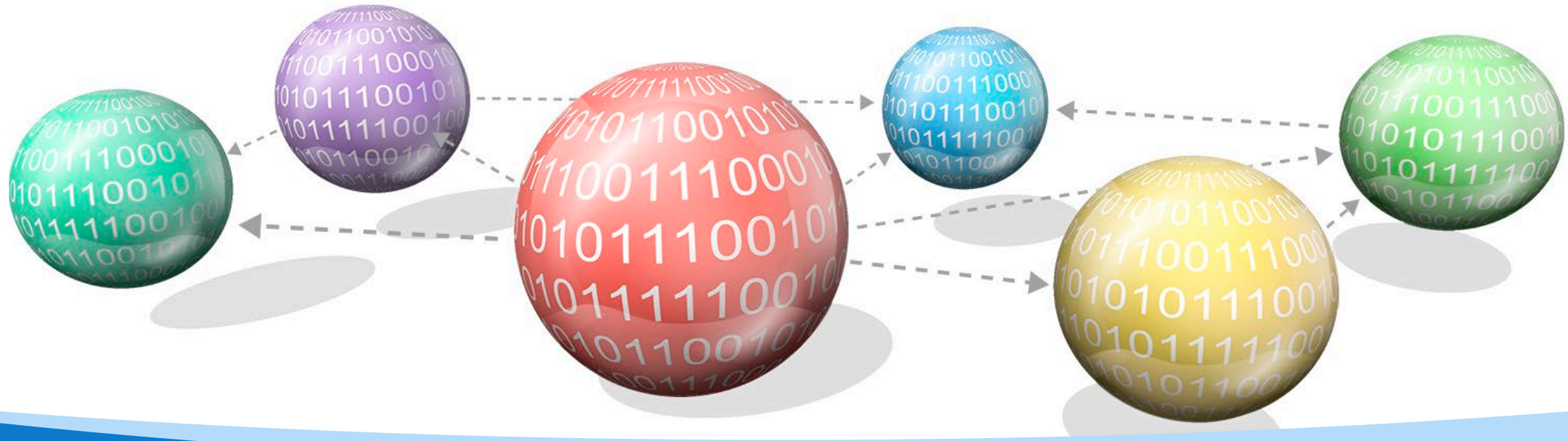


CZ/CE2002 Object-Oriented Design & Programming

Chapter 6: UML Model Class Relationships - Class Diagram

Dr. Li Fang
Senior Lecturer, College of Computing and Data Science

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Why OODP:Real-world Modeling

- OODP is ideal for modeling real-world systems and relationships. Objects can represent tangible things, like users or products, or more abstract concepts, like processes or transactions.
- This modeling makes it intuitive for people to design and think about systems in a way that mirrors real-world scenarios.

Recap: object-oriented programming (OOP) concepts

- **Encapsulation and Abstraction**

- By bundling data and methods that operate on that data, OODP promotes encapsulation, which hides the internal workings of an object. This makes the system more secure and reduces complexity for developers using that object.

- Abstraction allows focusing on higher-level operations without needing to know the specifics of how the object works, improving both productivity and code clarity.

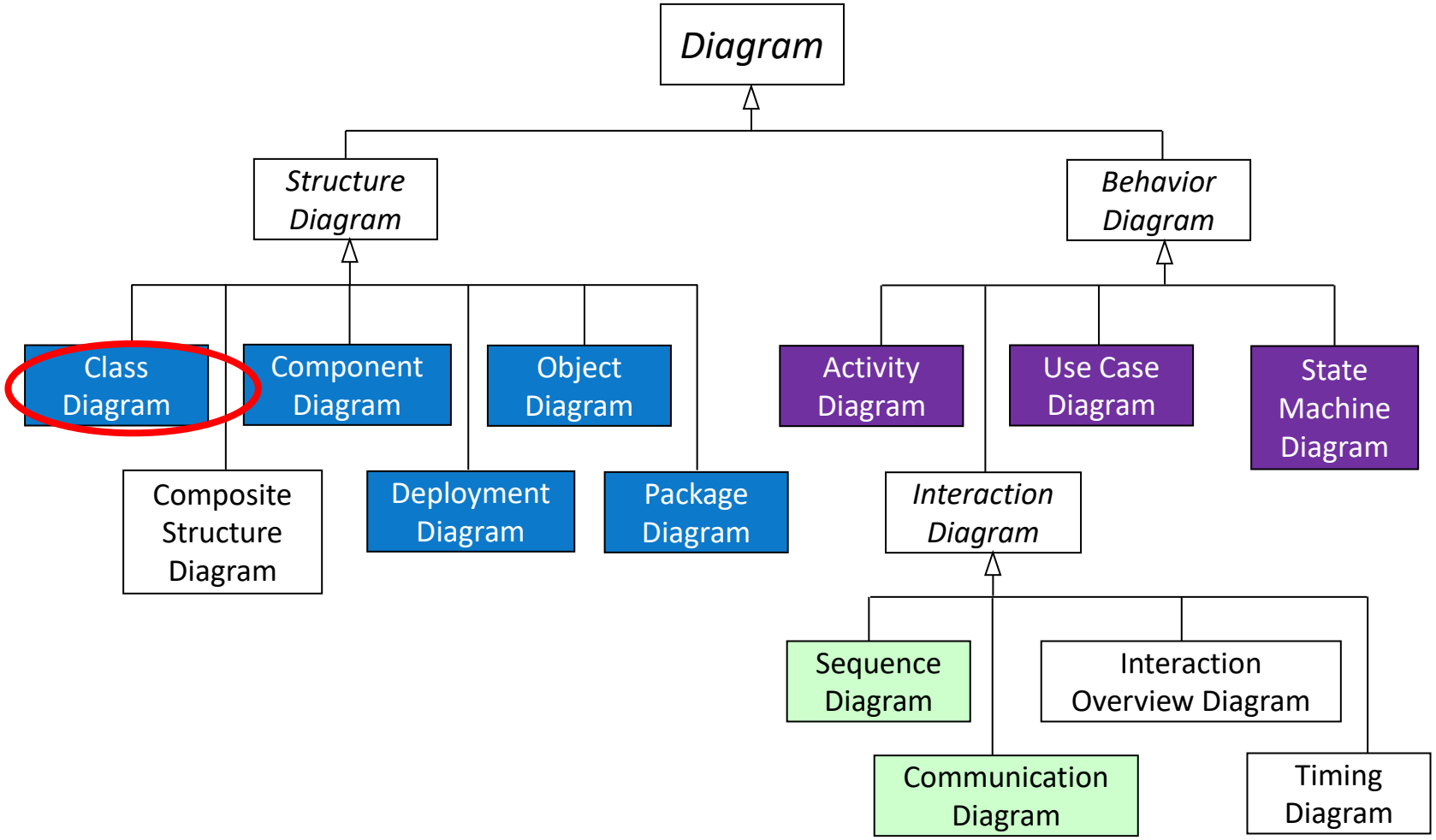
- **Polymorphism**

- Polymorphism allows objects to be treated as instances of their parent class, which is essential for flexible and scalable designs. This enables extending or updating systems without modifying existing code extensively.

object-oriented design (OOD) :Unified Modeling Language (UML)

- Overview of **Unified Modeling Language (UML)** and its use in OOD.
- Creating class diagrams to represent object-oriented designs.
- Understanding associations, generalizations, and multiplicity in UML.

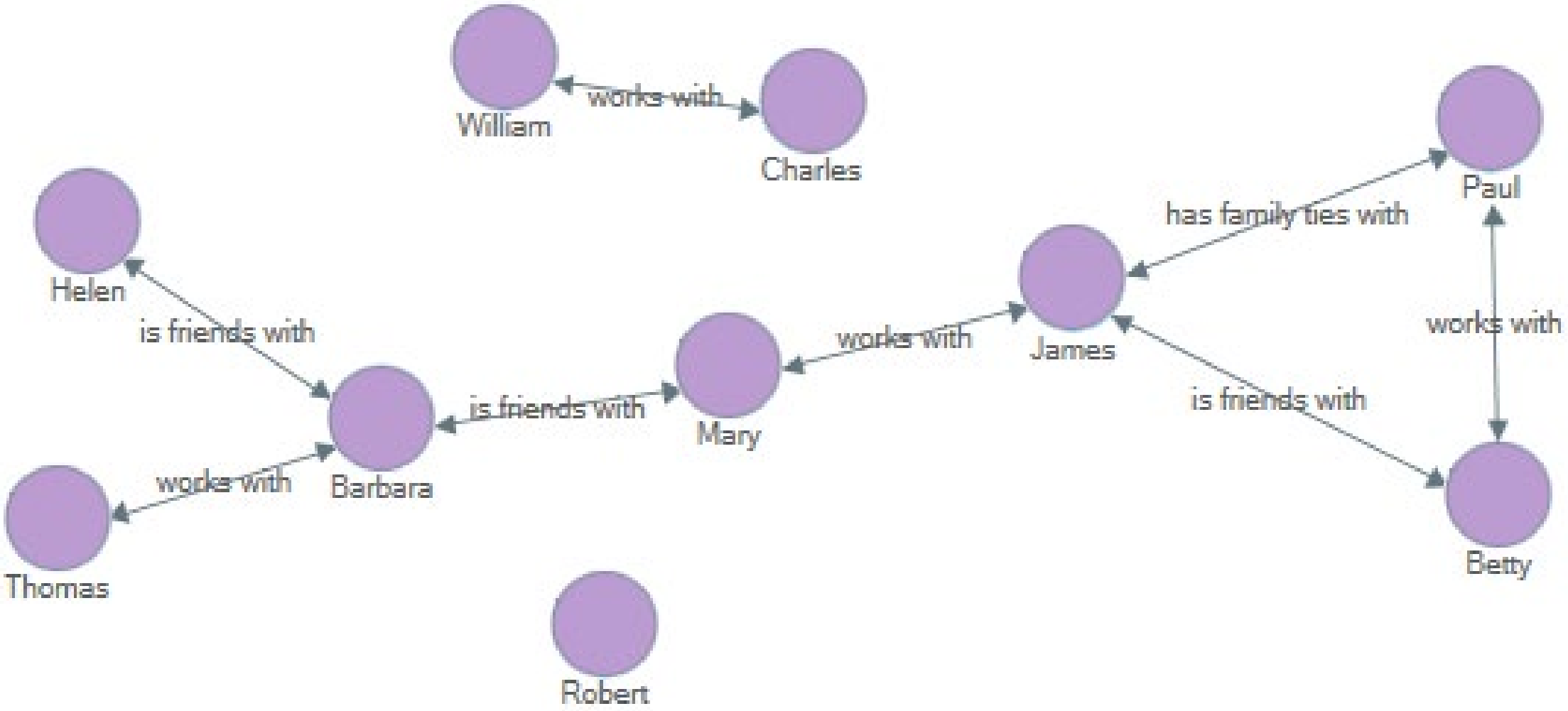
UML Diagram Types



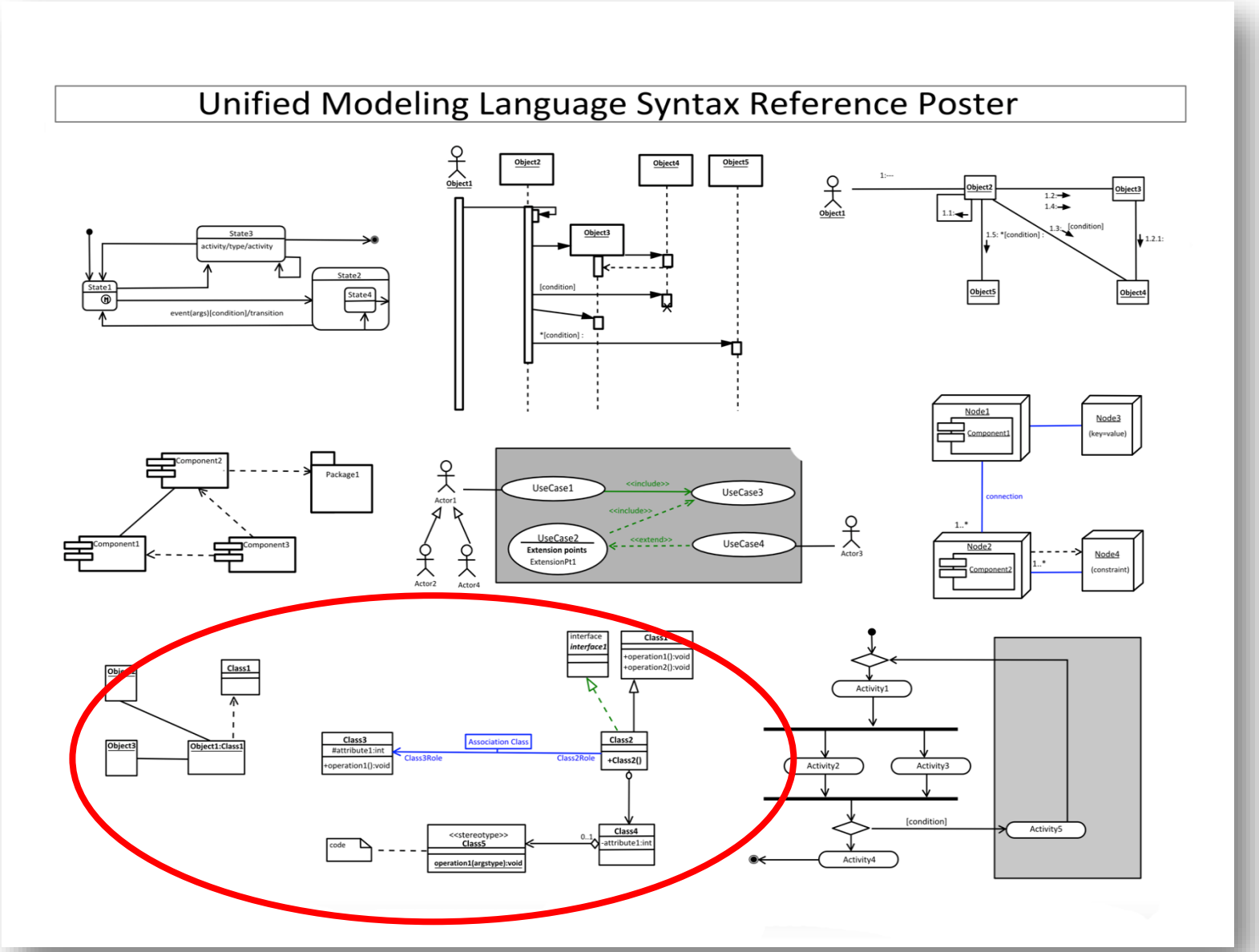
UML Class diagram

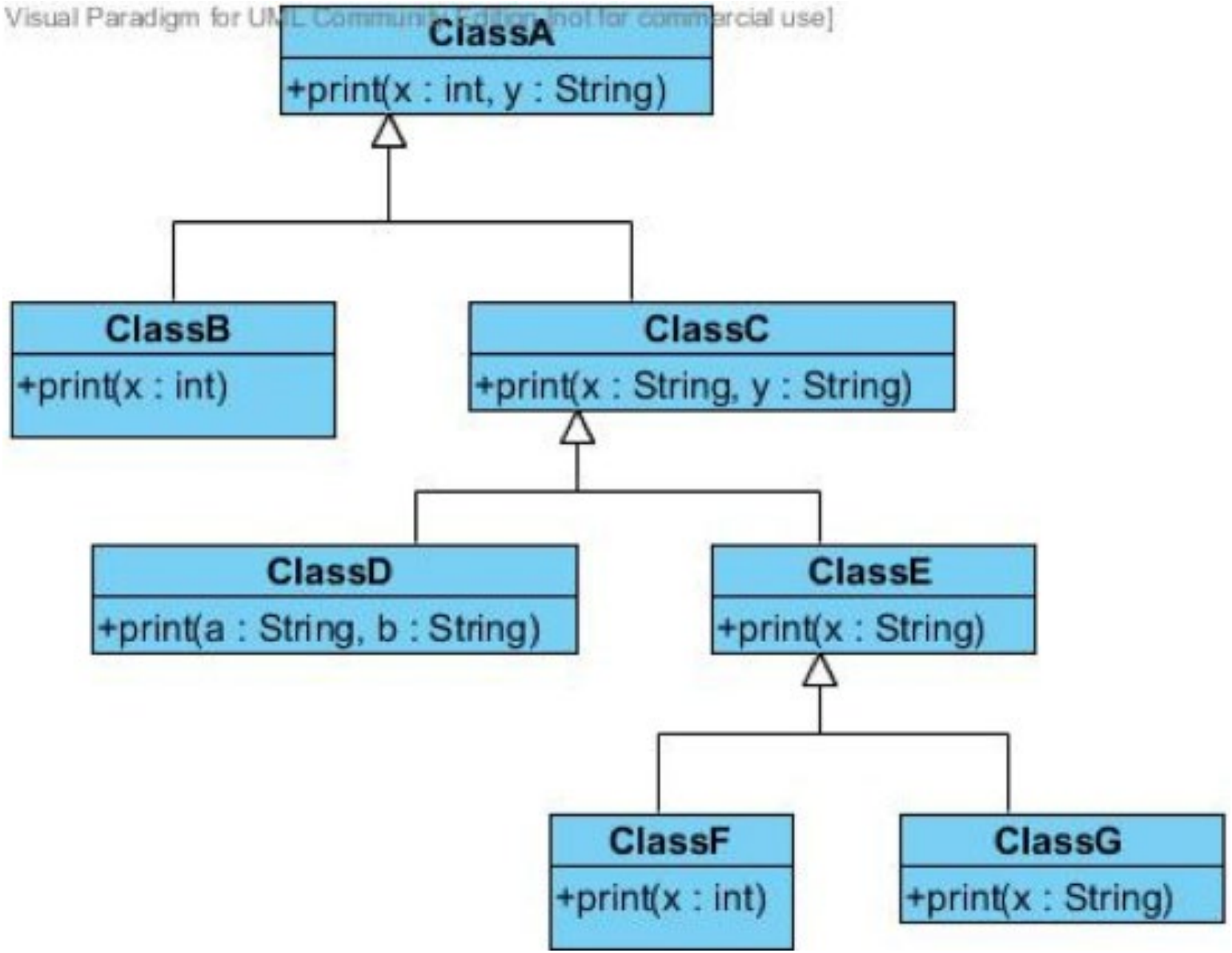
The UML Class diagram is a graphical notation used to construct and visualize object oriented systems. A class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's:

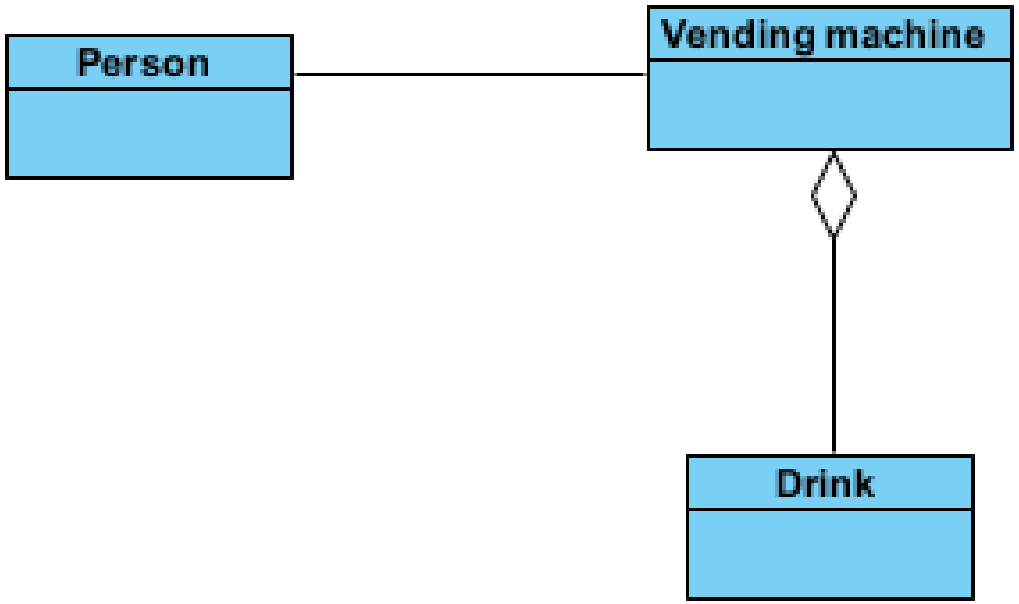
- classes,
- their attributes,
- operations (or methods),
- and the relationships among objects.


William
works with
Charles
Paul
has family ties with
Helen
is friends with
James
works with
Mary
is friends with
Barbara
works with
Thomas
is friends with
Betty
Robert

UML Diagram Types



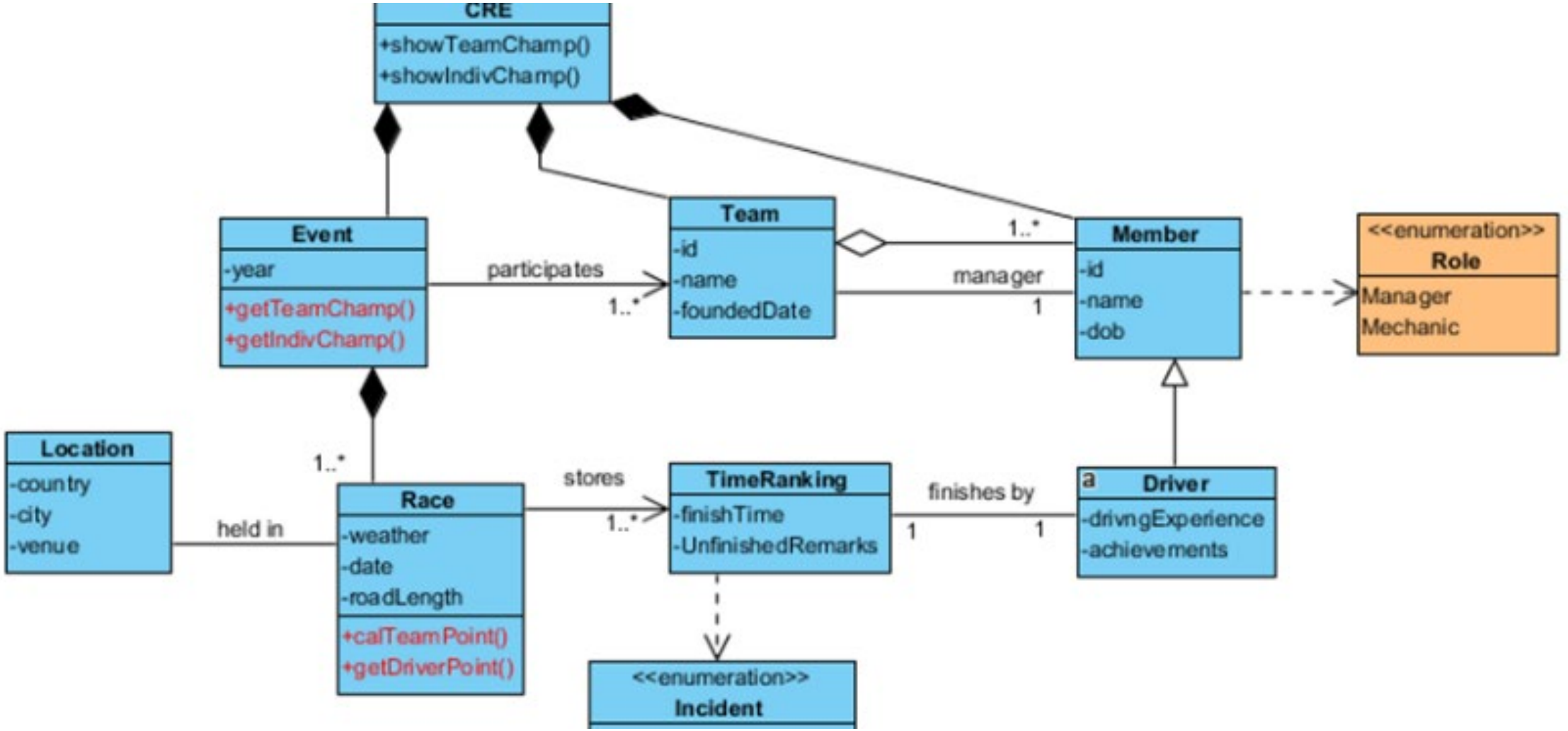

Visual Paradigm for UML Community Edition [not for commercial use]
ClassA
+print(x : int, y : String)
ClassB
+print(x : int)
ClassC
+print(x : String, y : String)
ClassD
+print(a : String, b : String)
ClassE
+print(x : String)
ClassF
+print(x : int)
ClassG
+print(x : String)


Person
Vending machine
Drink

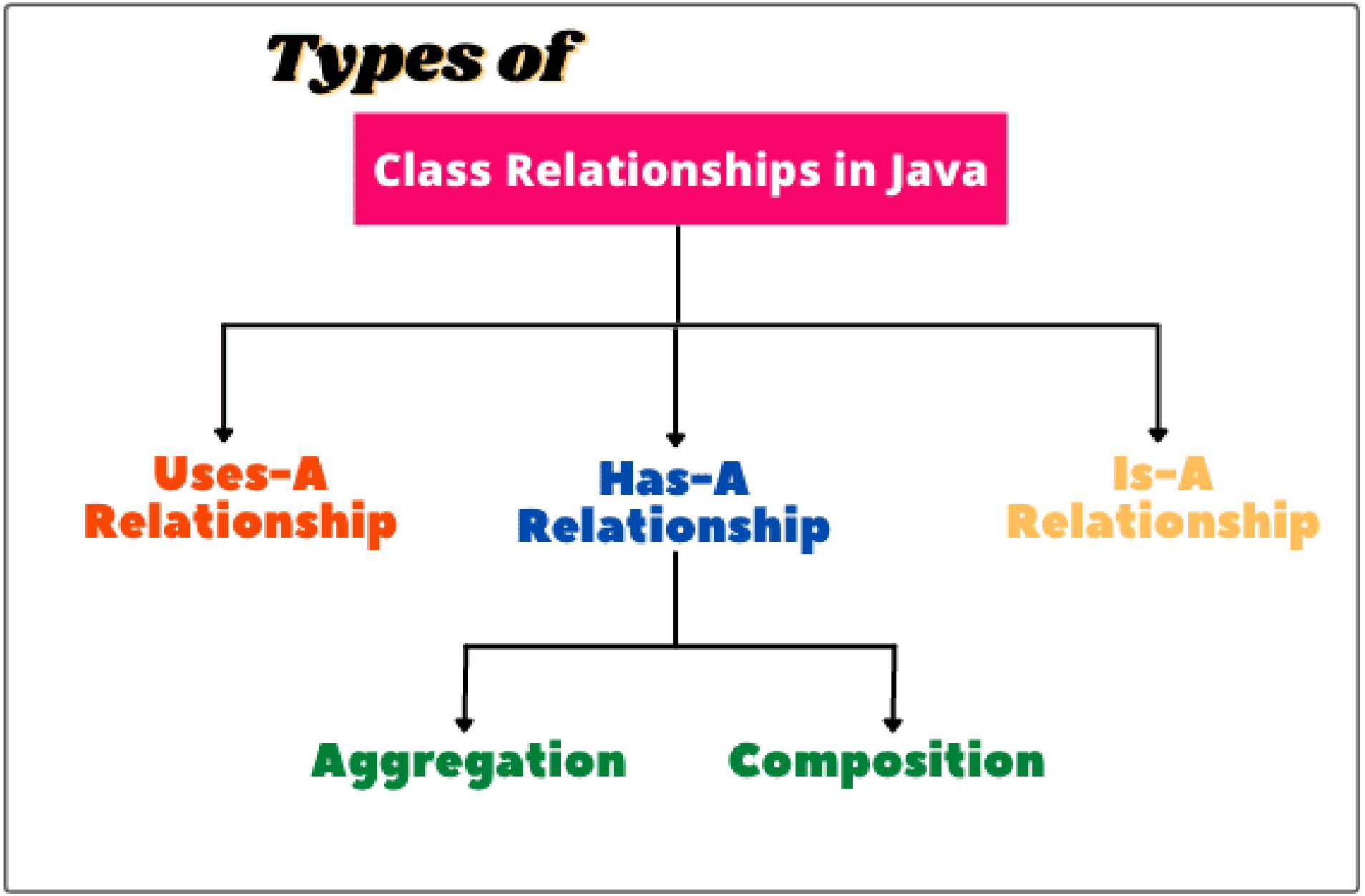

snack good
do good


1ST SGQR VENDING MACHINE
Powered by
dcube pay
In Partnership With

Final product



Topic 1: Relationship


Types of
Class Relationships in Java
Uses-A Relationship
Has-A Relationship
Is-A Relationship
Aggregation
Composition

Inheritance Hierarchy

Superclass
(parent class or base class)

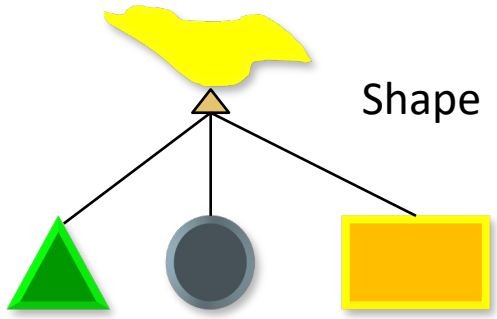
Super, base, parent, generalised

Shape

- xPos: double
- yPos: double

+ Shape()
+ Shape(xCoor: double, yCoor: double)
+ getXPos(): double
+ getYPos(): double
+ print(): void

Common attributes and methods



is-a

is-a

Rectangle

- width: double
- height: double

+ Rectangle()
+ Rectangle(xCoor: double,
yCoor: double,
w: double, h: double)
+ getWidth(): double
+ getHeight(): double
+ findArea(): double
+ findPerimeter(): double
+ print(): void

Additional attributes and methods

Circle

- radius: double

+ Circle()
+ Circle(xCoor: double, yCoor: double
rad: double)
+ getRadius(): double
+ findArea(): double
+ findPerimeter(): double
+ print(): void

Sub, derived, child, specialised

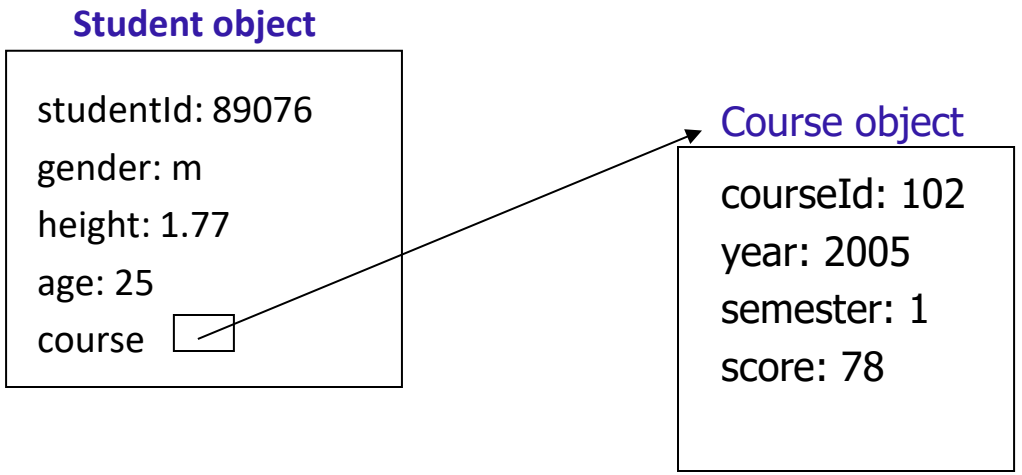
Subclasses: Inherits all the instance variables and methods of the superclass.

(EXCEPT those declared as private - accessibility)

Object Composition

- An object can include other objects as its **data** member. This is called **object composition**.
- A class contains object references from other classes as its instance variables.
- Object references are of reference data types.
- **In OO, this is called the "has-a" relationship.**
- Example: a student *has a* course.

```
public class Student {
   private int    studentId;
   private char   gender;
   private double height;
   private int    age;
   private Course course;
```




```
public class Course {
   private int courseId;
   private int year;
   private int semester;
   private int score;
```

a. Dependency (“Uses-A”)
b. Association (“Has-A”)
c. Inheritance (“Is-A”)

Use a →normal method calling
Has a →object composition
Is a →super class, child class

Topic 1.1: Association

Has-a

- **Association:** A generic relationship where one class uses or interacts with another. It represents a "has-a" relationship without implying strong ownership or lifecycle control. (e.g., A teacher teaches students.)
- **Aggregation:** A special type of association that represents a "whole-part" relationship but with weaker ownership. The lifecycle of the part is independent of the whole. (e.g., A library contains books, but books can exist without the library.)
- **Composition:** A stronger form of aggregation where the "whole" has full ownership of its parts, and the parts cannot exist independently. (e.g., A house is composed of rooms, and the rooms do not exist independently of the house.)

Student

TutorialGroup

slido

Which of the following statement(s) describe(s) the relationship between student and tutorial group correctly?

ⓘ Start presenting to display the poll results on this slide.

Which of the following statement describes the relationship between student and tutorial group correctly?

A. A student is a special tutorial group

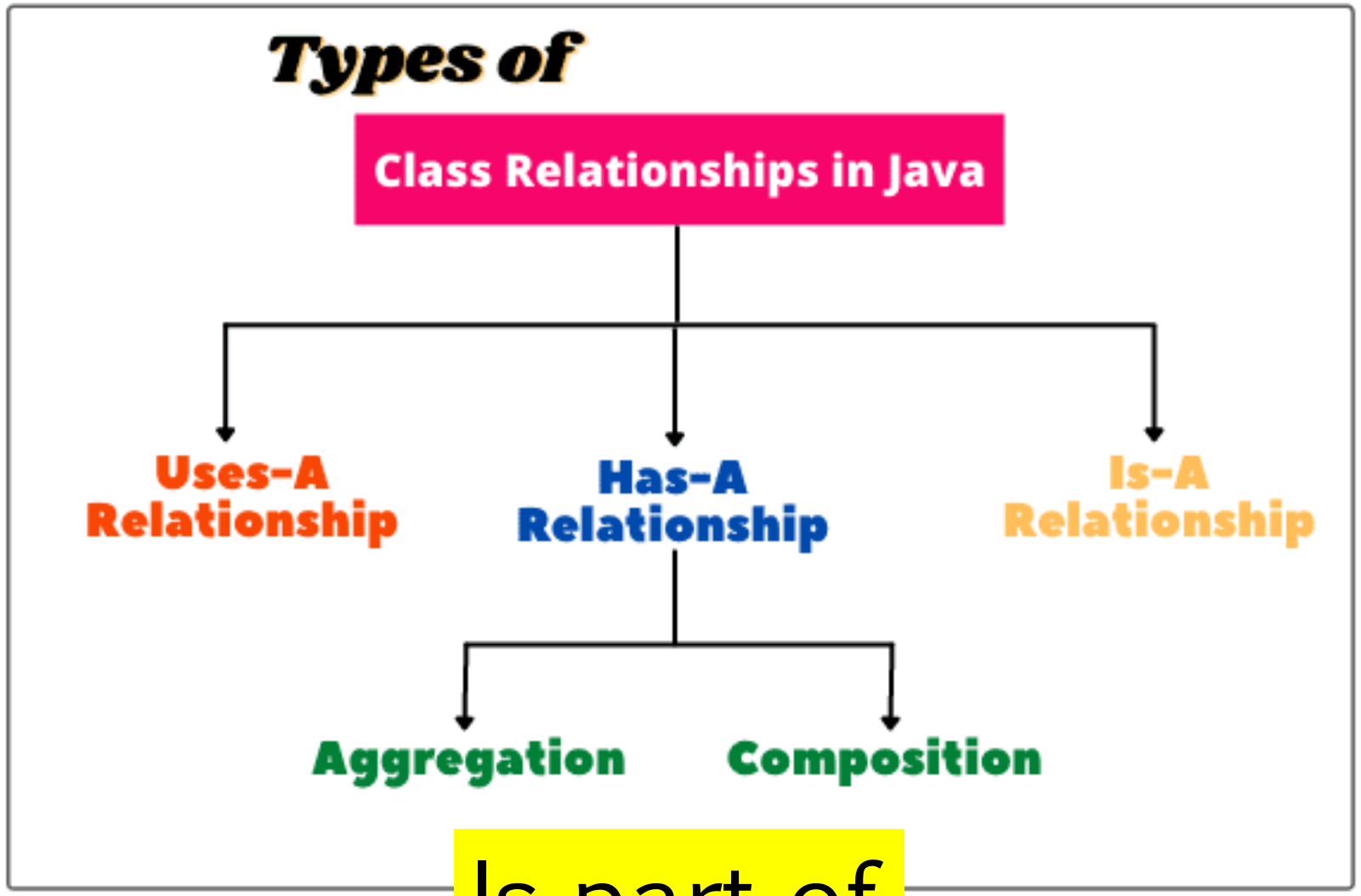
B. A student uses a tutorial group

C. A tutorial group has a student/students

D. A student is part of a tutorial group

E. All of above

F. None of above



Is part of

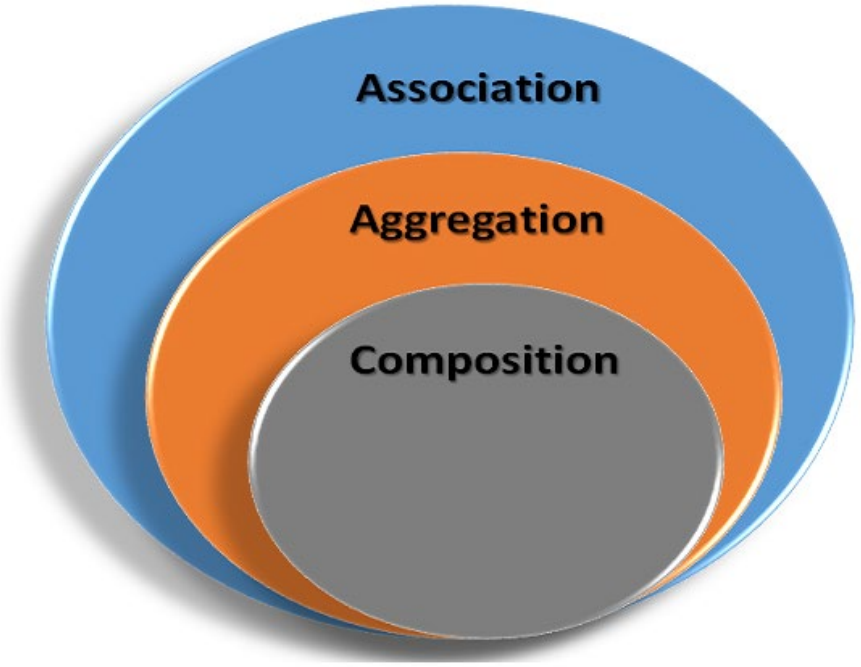
A has a B

B is part of A

slido

How do you describe the relationship between a person's hands and the person?

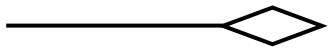
ⓘ Start presenting to display the poll results on this slide.

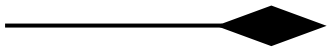


- Association is a relationship between two objects.
- Aggregation is a special form of association.
- Composition is a special form of aggregation.

AGGREGATION VERSUS COMPOSITION

AGGREGATION	COMPOSITION
An association between two objects which describes the "has a" relationship	The most specific type of aggregation that implies ownership
Destroying the owning object does not affect the containing object	Destroying the owning object affects the containing object
Diamond symbol represents the aggregation in UML	Highlighted diamond symbol represents the composition in UML





slido

Based on your understanding, what is the relationship between TutorialGroup and Student?

ⓘ Start presenting to display the poll results on this slide.

Based on your understanding, what is the relationship between TutorialGroup and Student?

A. Inheritance

B. Aggregation

C. Composition

D. All of above

E. None of above

slido

Based on your understanding, what is the relationship between Brain and Person?

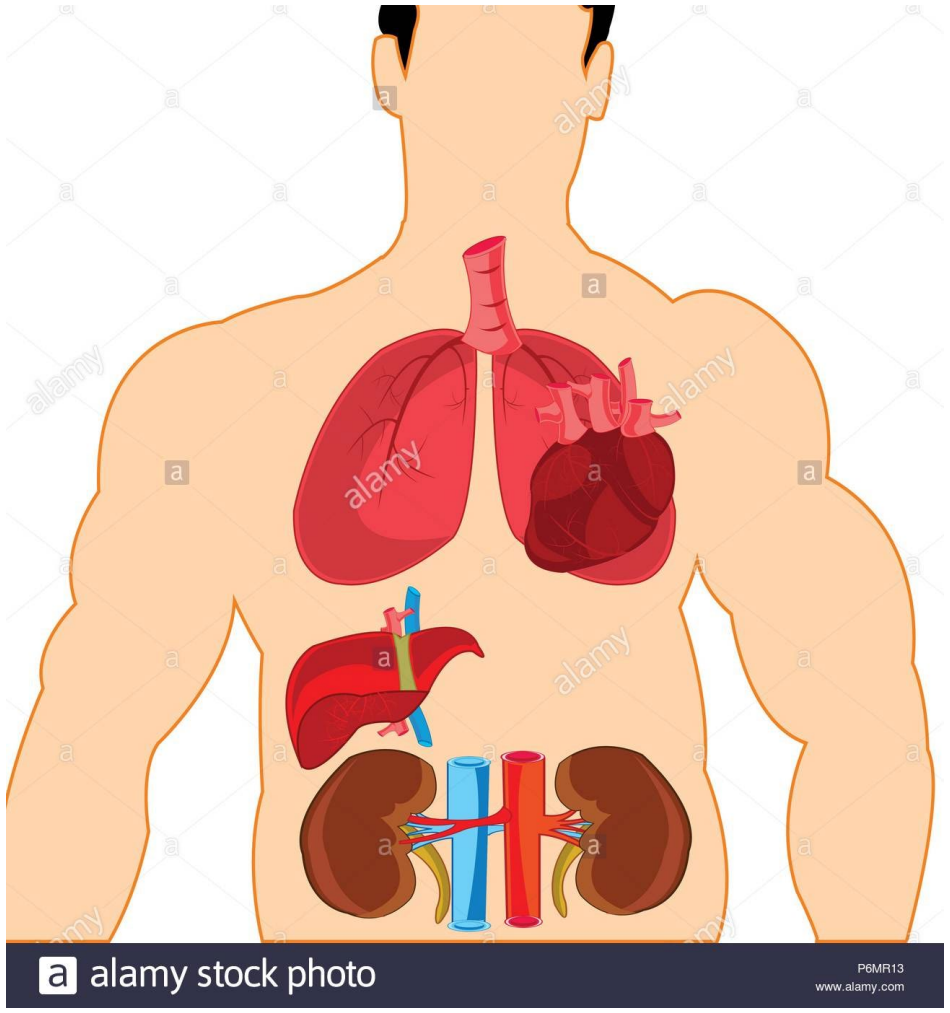
ⓘ Start presenting to display the poll results on this slide.

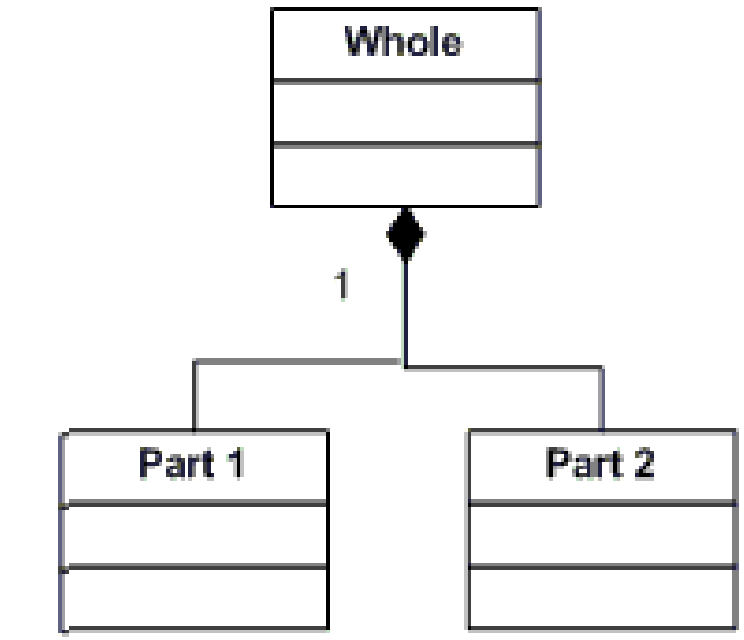
Based on your understanding, what is the relationship between Brain and Person?

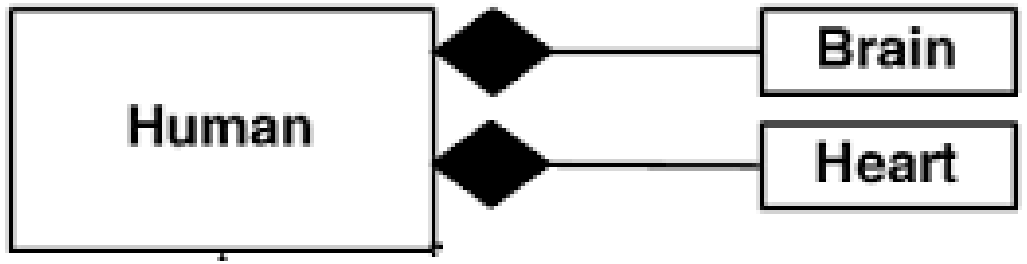
A. Inheritance
B. Aggregation

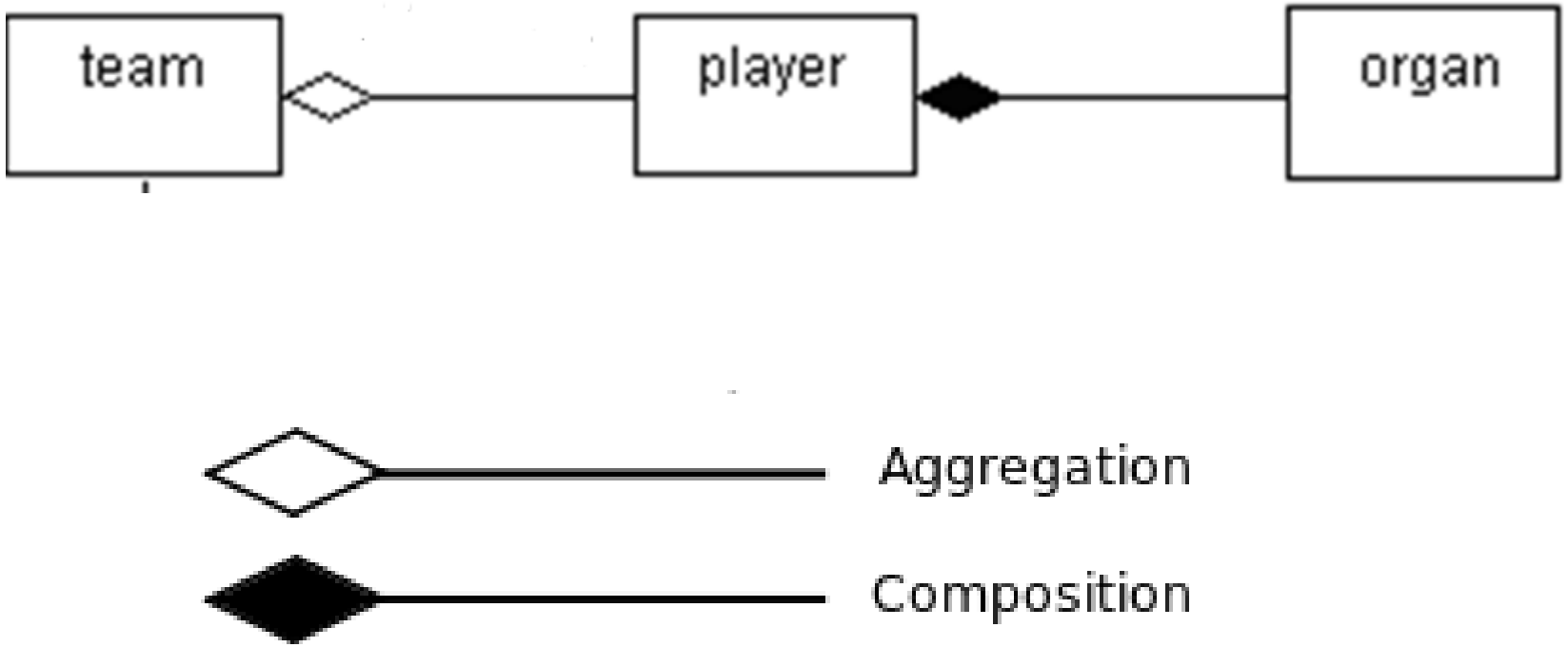
C. Composition
D. All of above
E. None of above

Best example of composition (No transplant)



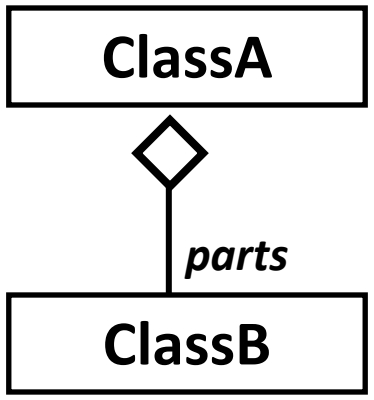





team
player
organ
Aggregation
Composition

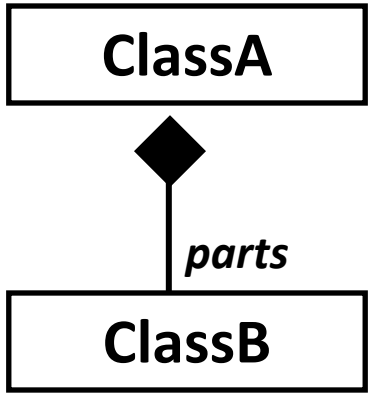
UML Class Diagram to Java Code

Aggregation




```
class  ClassA  {
      Vector  parts = new Vector() ;  // attribute
      ……..  // parts stores a collection of ClassB objs
      public void addClassB(ClassB  b ) {
            ……..// added through reference
               parts.add(b) ;
      }
     ……..……..
} // use of Vector class is just to show dynamic (*) Collection,
//  can be ArrayList, etc OR array[ ] (if size is fixed)
```

Composition



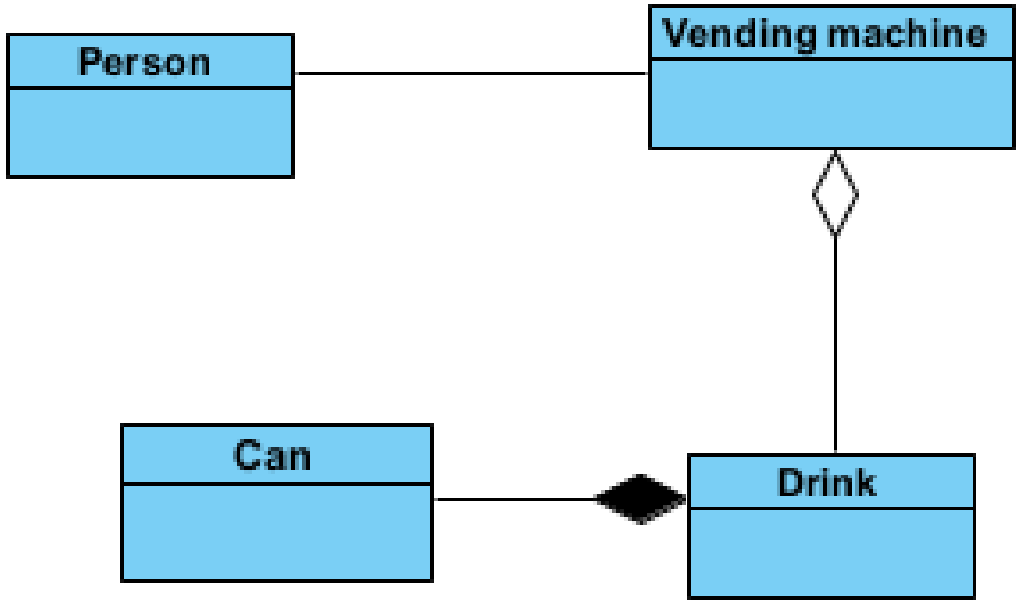

```
class  ClassA  {
      Vector  parts = new Vector();  // attribute
      ……..
      public ClassA( ) {
            ClassB  b1 = new ClassB() ;  // created in class
               ClassB b2 = new ClassB() ;
               parts.add(b1) ;
               parts.add(b2) ;
      }
     ……..……..
}
```

slido

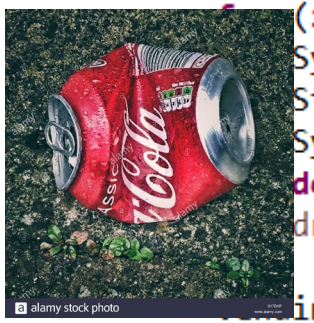
Based on your understanding, what is the relationship between can drinks and a vending machine?

ⓘ Start presenting to display the poll results on this slide.

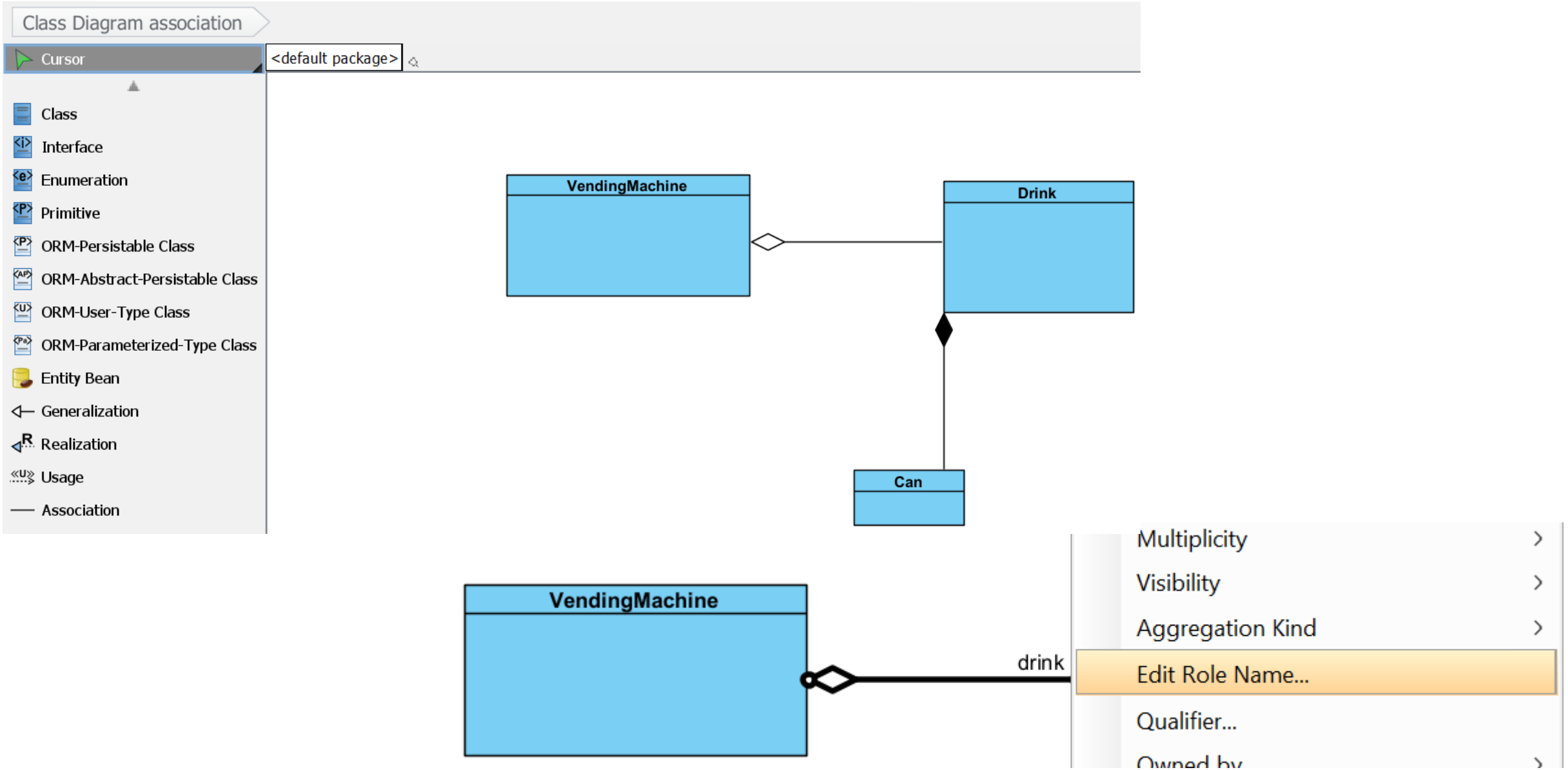
```java
private Drink[] drinks ; // object composition - relevant

public VendingMachineImprove(Drink[] drinks) {
  this.drinks = drinks;
}
```



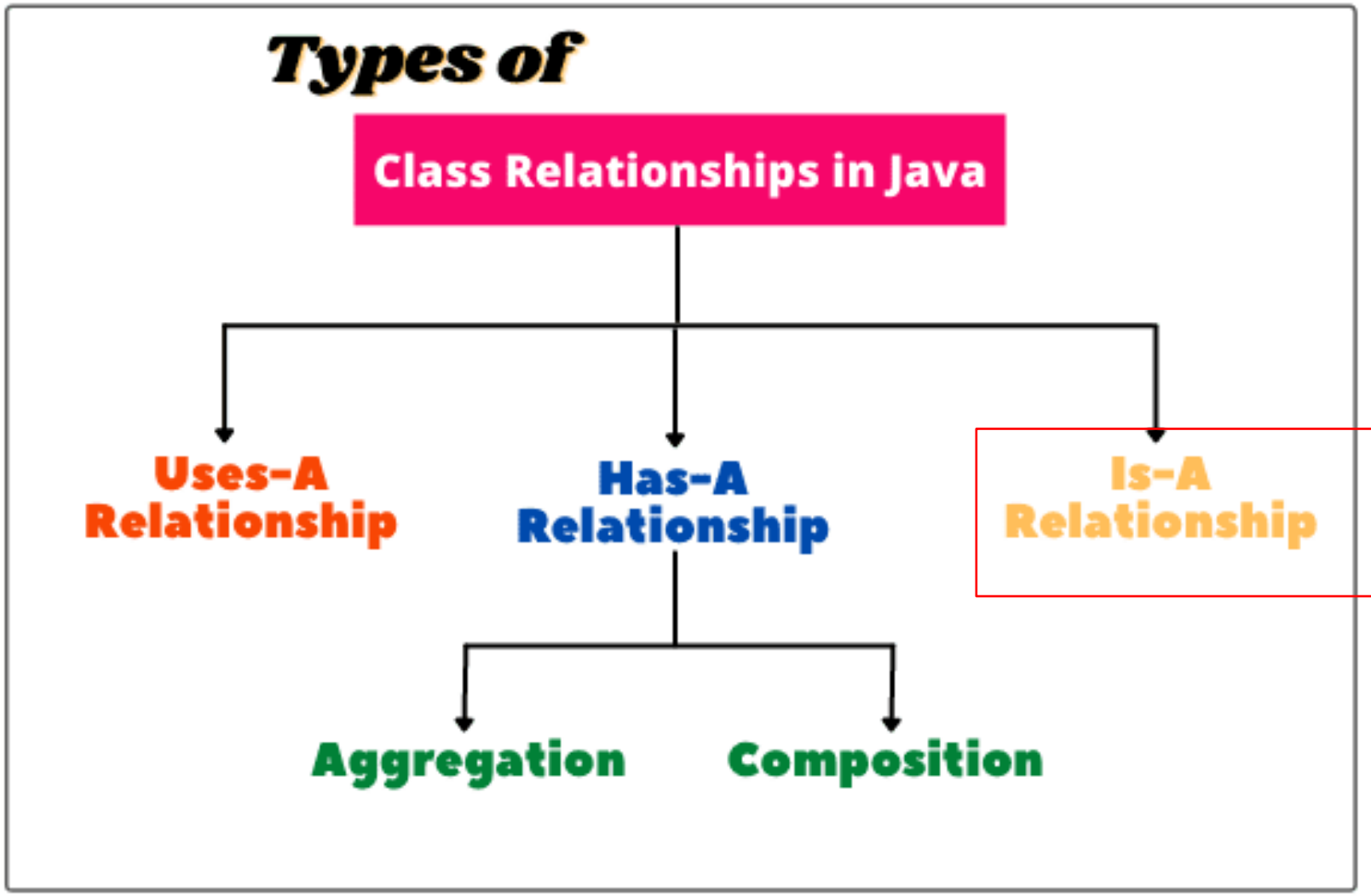

```java
import java.util.Scanner;
public class VendingMachineAppImprove {
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Settings....");
        // future improvement can just read from file or DB
        System.out.println("Please enter number of drink types: ");
        int drinkTypes = sc.nextInt();
        Drink[] drinks = new Drink[drinkTypes];
        for(int i =0; i<drinkTypes; i++ ){
            System.out.println("Please enter drink name: ");
            String drinkName = sc.next();
            System.out.println("Please enter drink price: ");
            double drinkPrice = sc.nextDouble();
            drinks[i] = new Drink(drinkName,drinkPrice) ;

        }
        VendingMachineImprove vm = new VendingMachineImprove(drinks) ;
        vm.start() ;
    }
}
```



Demo

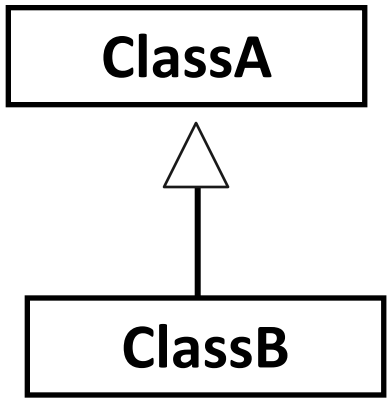

Class Diagram association
Cursor
<default package>
Class
Interface
Enumeration
Primitive
ORM-Persistable Class
ORM-Abstract-Persistable Class
ORM-User-Type Class
ORM-Parameterized-Type Class
Entity Bean
Generalization
Realization
Usage
Association
VendingMachine
Drink
Can
VendingMachine
drink
Multiplicity
Visibility
Aggregation Kind
Edit Role Name...
Qualifier...
Owned by

Topic 1.2: Inheritance

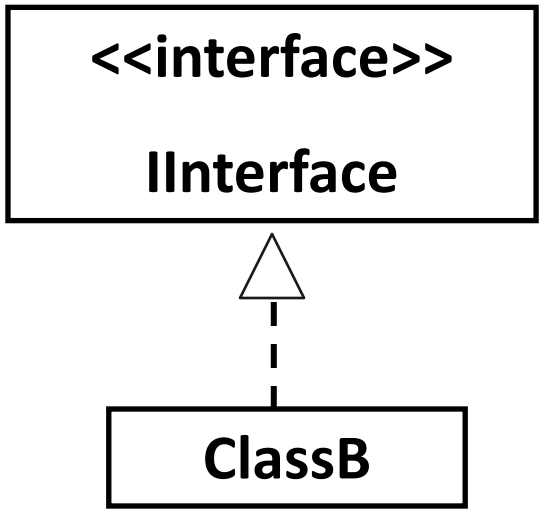

Types of
Class Relationships in Java
Uses-A Relationship
Has-A Relationship
Is-A Relationship
Aggregation
Composition

UML Class Diagram to Java Code

Generalisation (Inheritance)



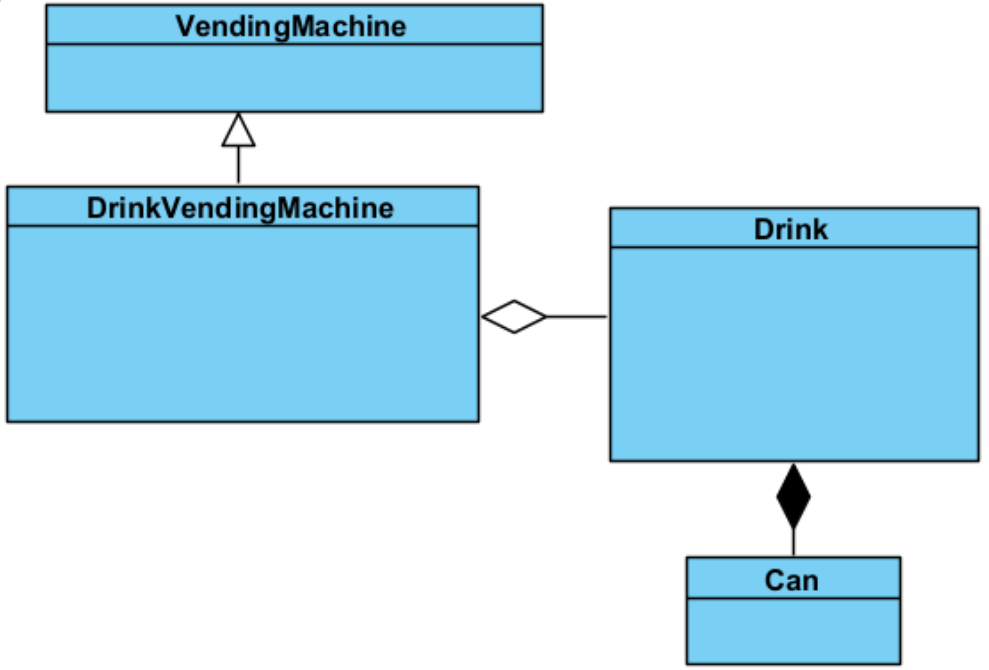

```
class  ClassB extends ClassA {
.....................
}
```

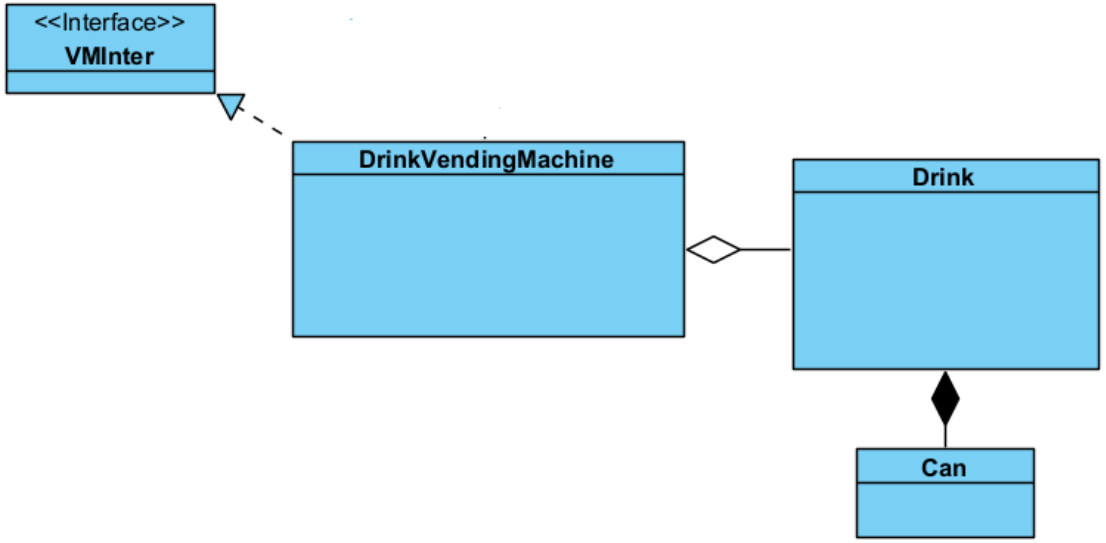


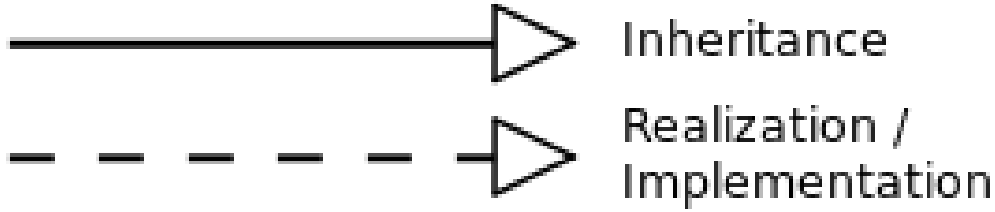

```
class  ClassB  implements  IInterface{
.....................
}
```

Realisation

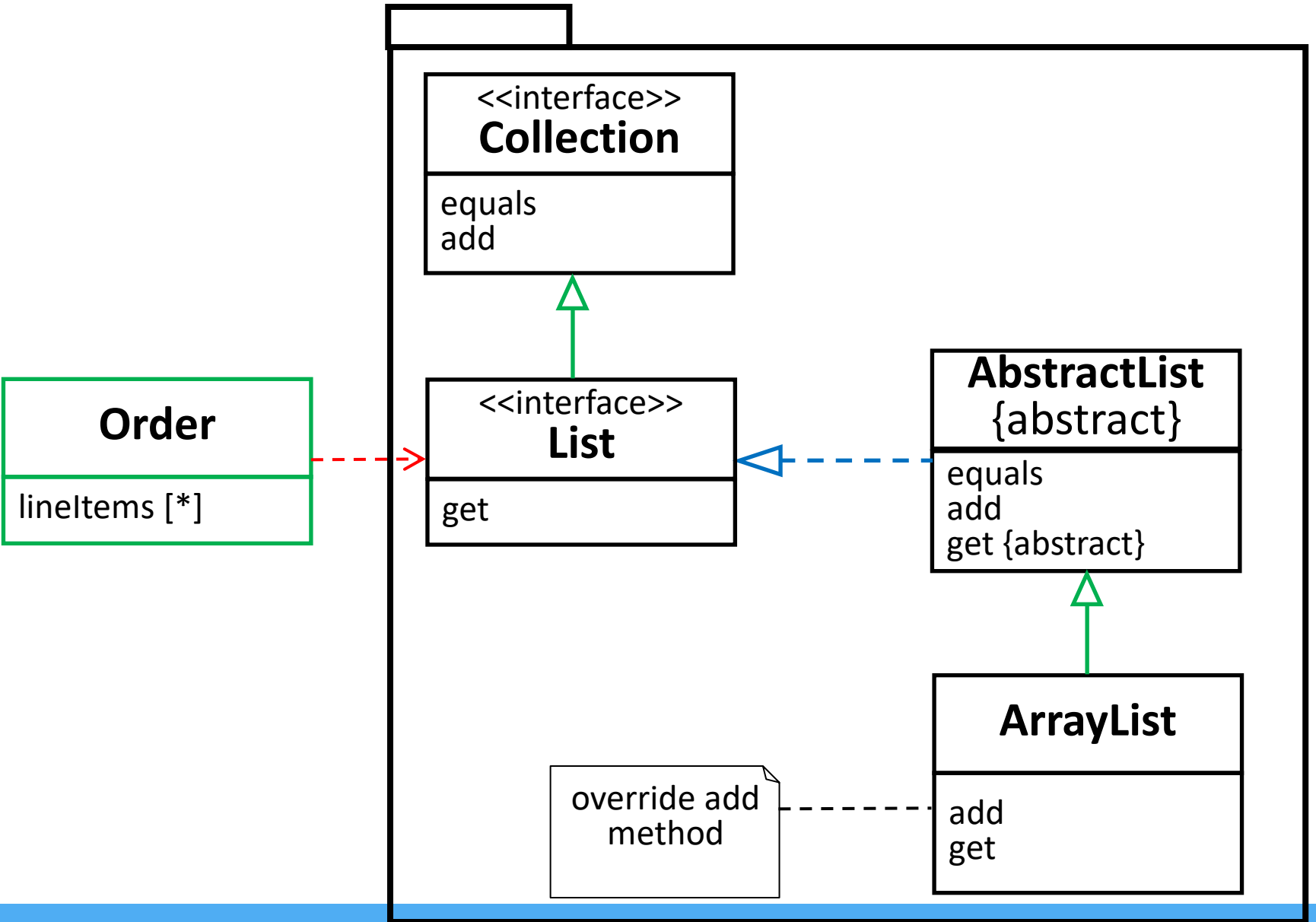

FRESH&HOT
POPCORN
TO GO


VendingMachine
DrinkVendingMachine
Drink
Can


<<Interface>>
VMInter
DrinkVendingMachine
Drink
Can


Inheritance
Realization /
Implementation

Java Collections Framework

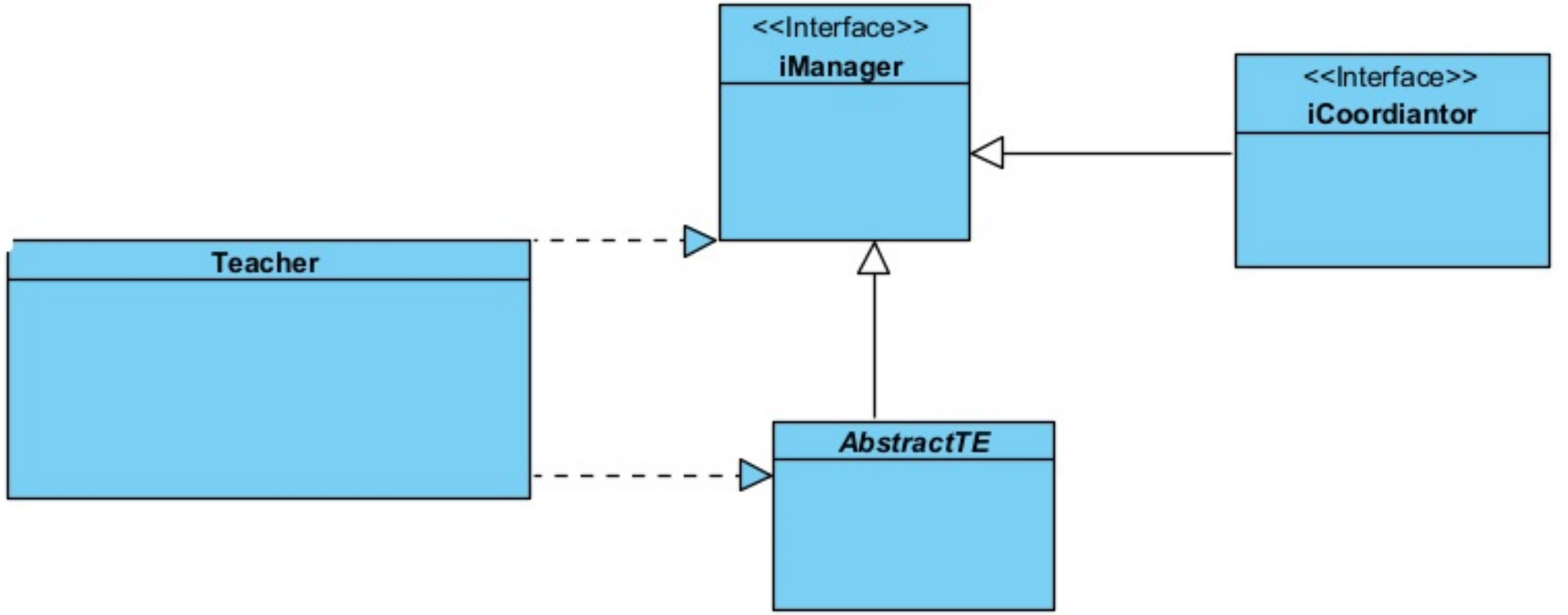


- Interface inherits Interface,
- implementation only happen when class inherits interface.
- Abstract class are classes.

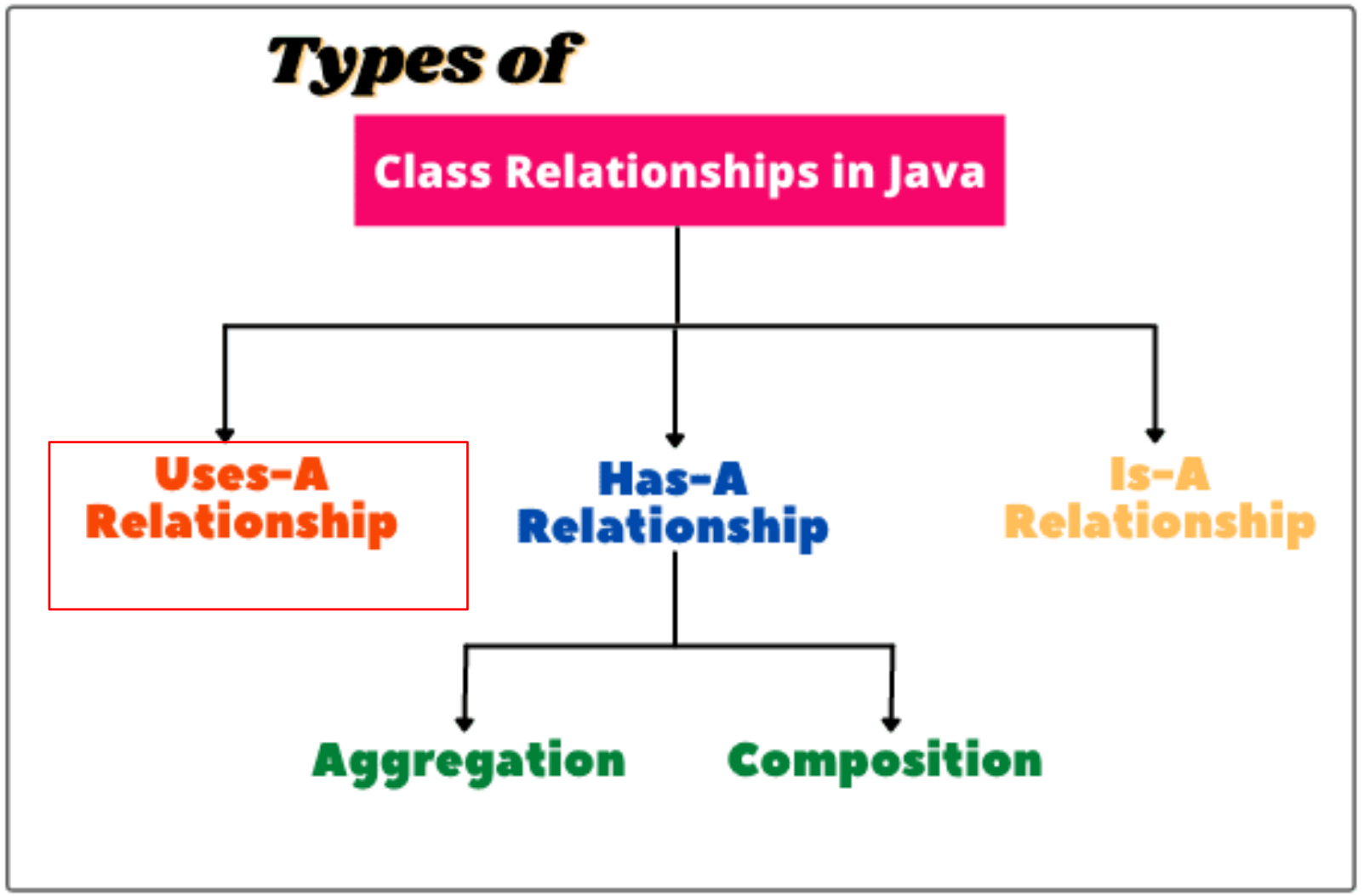
slido

What are the errors in the class diagram?

ⓘ Start presenting to display the poll results on this slide.

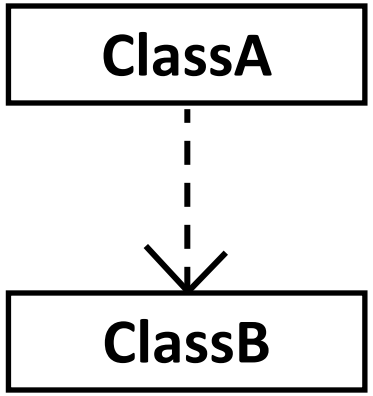

<<Interface>>
iManager
<<Interface>>
iCoordiantor
Teacher
AbstractTE

Topic 1.3: Dependency


Types of
Class Relationships in Java
Uses-A Relationship
Has-A Relationship
Is-A Relationship
Aggregation
Composition

UML Class Diagram to Java Code

Dependency




```
class  ClassA  {
      .......
       ClassB  doSomeThing(...)  {  // as return type
       ........
       .....
       }
}
```

```
class  ClassA  {
      .......
       <ReturnType>  doSomeThing(ClassB b, ...)  {  // as method argument
       ........
       .....
       }
}
```

```
class  ClassA  {
      .......
       <ReturnType>  doSomeThing(......)  {
              ClassB b = new ClassB() ; // as variable inside method
               .....
       }
}
```

They (Dependency Relationships) are not implemented with member variables at all.

slido

Based on the program below, there are four statements, Which of the following statement(s) is/are correct?

ⓘ Start presenting to display the poll results on this slide.

Based on the program below, there are four statements, Which of the following statement(s) is/are correct?

```
public class A {
    private C c;
    public void myMethod(B b) {
        b.callMethod();
    }
}
```

I. Relationship between A and B is Association
II. Relationship between A and C is Association
III. Relationship between A and B is Dependency
IV. Relationship between A and C is Dependency

A. I and II

B. I and IV

✓ C. II and III

D. III and IV

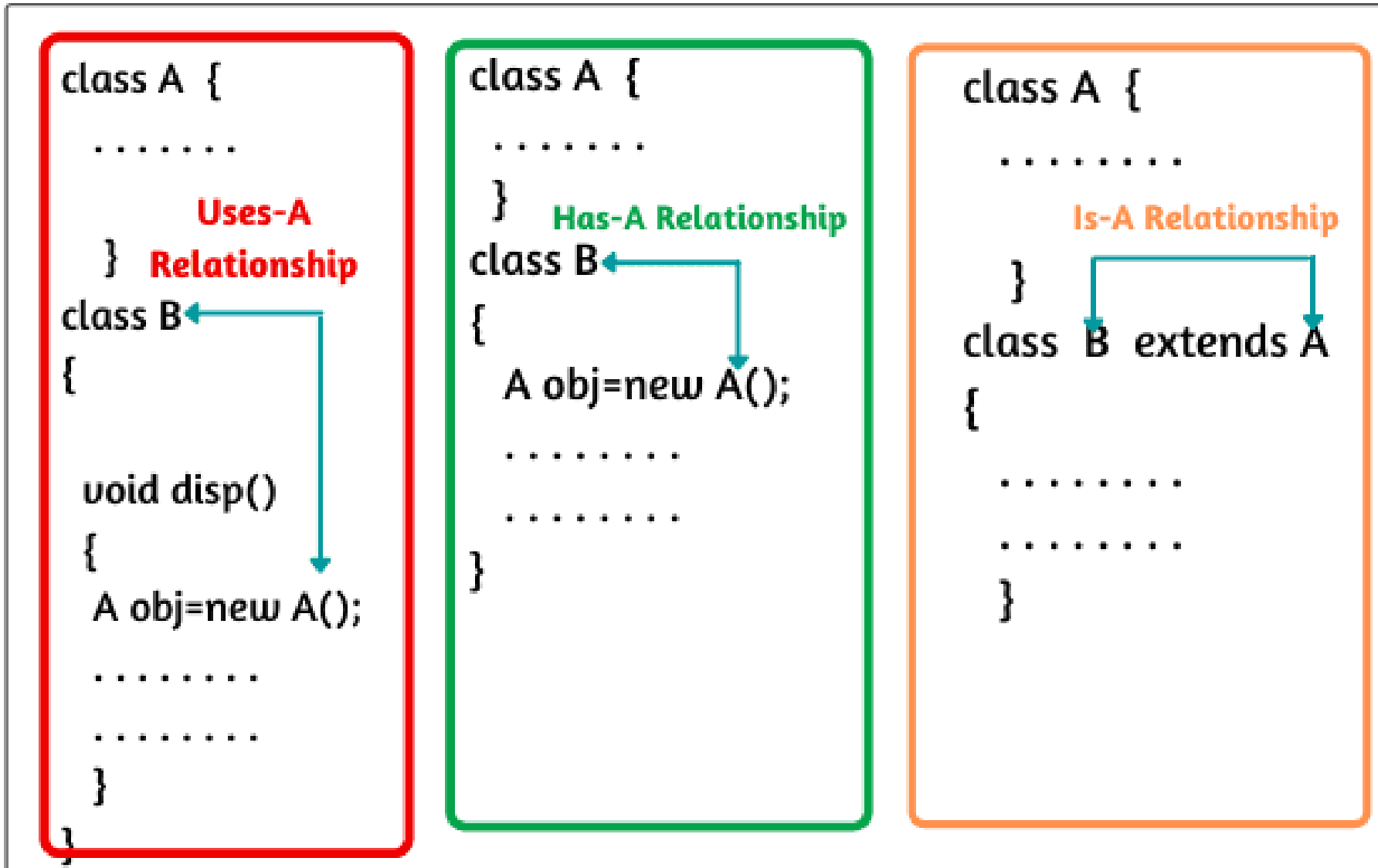
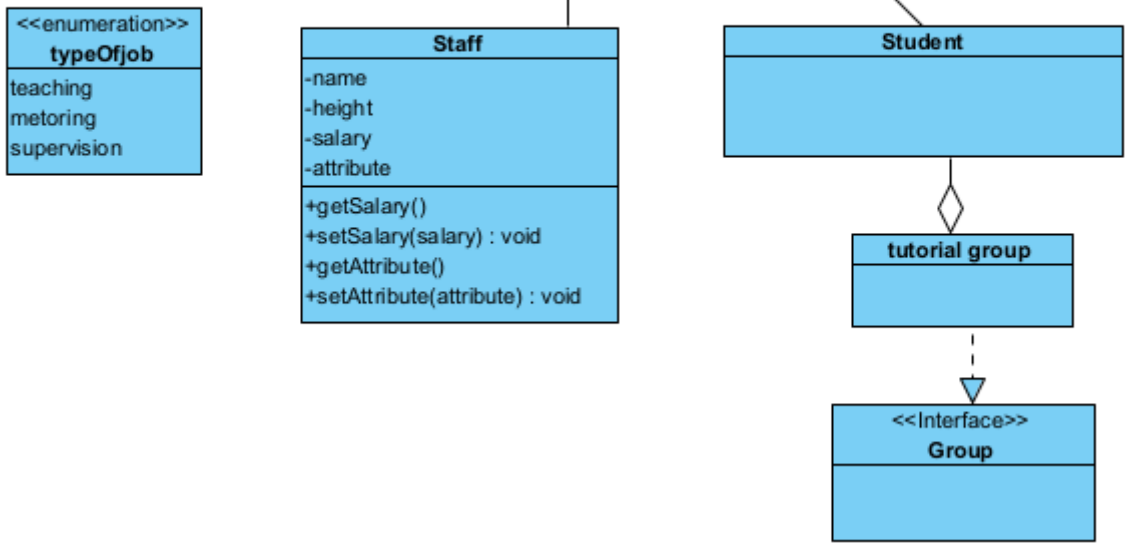
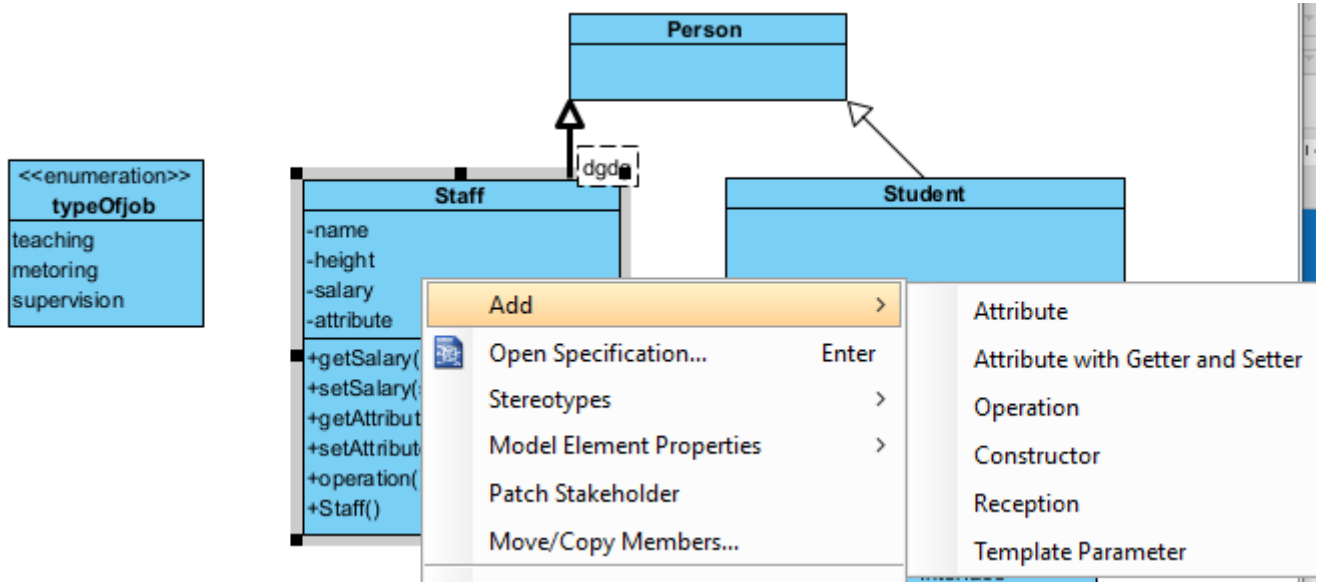
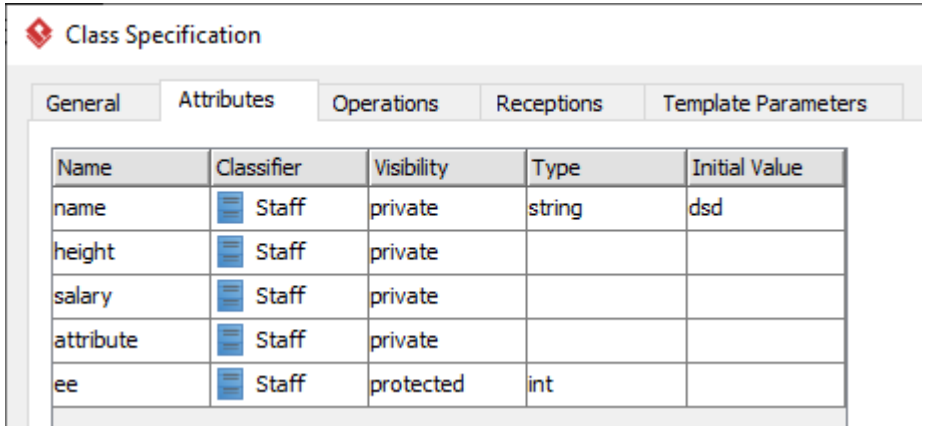


Fig: Different forms of relationship between classes in Java

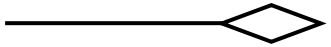
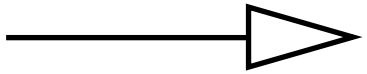
Demo


<<enumeration>>
typeOfjob
teaching
metoring
supervision
Person
Staff
-name
-height
-salary
-attribute
+getSalary()
+setSalary(salary) : void
+getAttribute()
+setAttribute(attribute) : void
Student
tutorial group
<<Interface>>
Group


<<enumeration>>
typeOfjob
teaching
metoring
supervision
Person
Staff
-name
-height
-salary
-attribute
Student
Add
Open Specification... Enter
Stereotypes
Model Element Properties
Patch Stakeholder
Move/Copy Members...
Attribute
Attribute with Getter and Setter
Operation
Constructor
Reception
Template Parameter


Class Specification
General
Attributes
Operations
Receptions
Template Parameters
Name | Classifier | Visibility | Type | Initial Value
name | Staff | private | string | dsd
height | Staff | private
salary | Staff | private
attribute | Staff | private
ee | Staff | protected | int

Summary

- Class Diagram Notations
- Type of Class relationships
  - Dependency 
    - Association 
      - Aggregation 
        - Composition 
    - Generalisation 
    - Realisation 

Dependency vs Association

```
class Teacher {
    String name;
    void teach(Student student) {
        System.out.println(name + " is teaching " + student.name);
    }}
```

```
class Student {
    String name;
}
```

```
public class Test {
    public static void main(String[] args)
        Teacher teacher = new Teacher();
        teacher.name = "Mr. Smith";

        Student student = new Student();
        student.name = "John";

        teacher.teach(student);
    }}
```

- Dependency means that one class temporarily uses another class, usually through a method or parameter.
- The classes are not linked for the long term.
- In the demo code, the Teacher class depends on Student to perform its teach method, but there's no persistent relationship between the two. The Student object is passed as an argument, meaning the Teacher depends on Student only while calling the teach method.
- In dependency, the relationship is limited to method parameters (e.g., passing a Student object to a teach method), and once the method execution is over, the relationship ends. The teacher cannot "remember" which students were taught, nor can they modify the state of the students later on.

Association

```java
import java.util.List;
import java.util.ArrayList;

class Teacher {
    String name;
    List<Student> students; // Association with Student (Teacher knows about students)

    Teacher() {
        students = new ArrayList<>(); // Initialize the list of students
    }

    void addStudent(Student student) {
        students.add(student); // Add a student to the teacher's list
    }

    void teach() {
        for (Student student : students) {
            System.out.println(name + " is teaching " + student.name);
        }
    }
}
```

```java
class Student {
    String name;
}
```

```java
public class Test{
    public static void main(String[] args) {
        Teacher teacher = new Teacher();
        teacher.name = "Mr. Smith";

        Student student1 = new Student();
        student1.name = "John";

        Student student2 = new Student();
        student2.name = "Jane";

        // Establish association by adding students to the teacher's list
        teacher.addStudent(student1);
        teacher.addStudent(student2);

        // Teacher teaches the students (association)
        teacher.teach();
    }
}
```

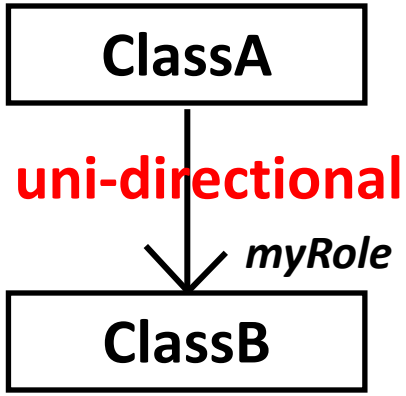
Key Takeaway:

- Dependency is short-term and based on method calls.
- Association is long-term and allows richer interactions and modifications over the lifecycle of the objects involved.

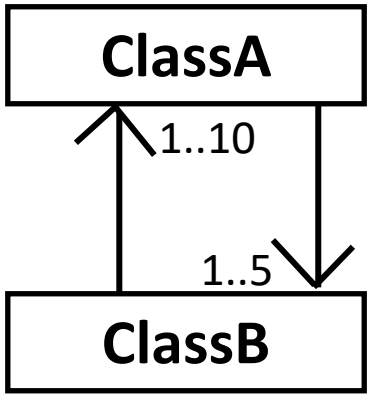
Topic 2: More on Association (uni-directional vs bi-directional)

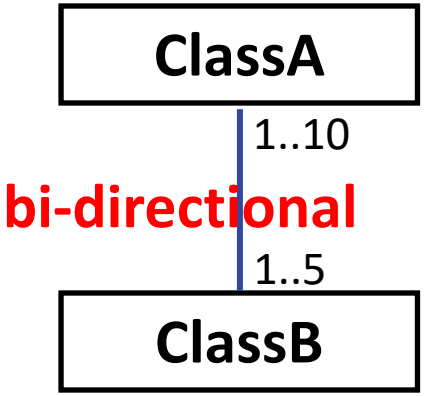
Association in Java Code

Association




```
class  ClassA  {
       ClassB  myRole ;  // attribute
       ……..
       …..
}
```





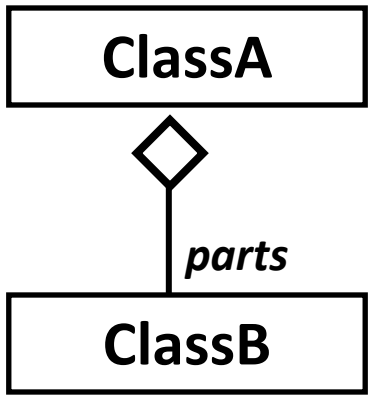

```
class  ClassA  {
       ClassB[ ] bObjs = new ClassB[ 5 ];  // attribute
       ……..
                    bObjs[0] = new ClassB(this) ;
……..
}
```

```
class  ClassB  {
   ClassA[ ] aObjs = new ClassA[ 10 ];  // attribute
       ……..
                public ClassB(ClassA a) { aObjs[0] =  a  ; …}
……..
}
```

Object composition

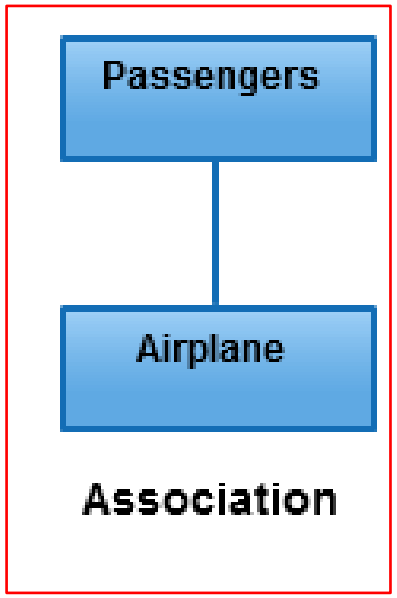
UML Class Diagram to Java Code

Aggregation

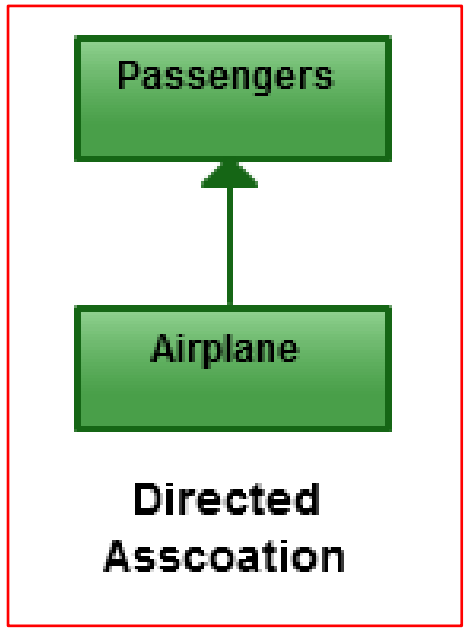



```
class  ClassA  {
        Vector  parts = new Vector() ;  // attribute
        ……..  // parts stores a collection of ClassB objs
        public void addClassB(ClassB  b ) {
              ……..// added through reference
             parts.add(b) ;
        }
       ……..……..
} // use of Vector class is just to show dynamic (*)
Collection,
//  can be ArrayList, etc OR array[ ] (if size is fixed)
```

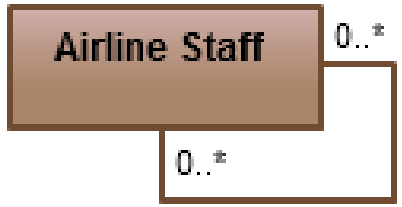
Aggregation denotes whole/part relationships whereas associations do not. However, there is not likely to be much difference in the way that the two relationships are implemented. That is, it would be very difficult to look at the code and determine whether a particular relationship ought to be aggregation or association.
– Robert C. Martin, UML Tutorial: Part 1 — Class Diagrams



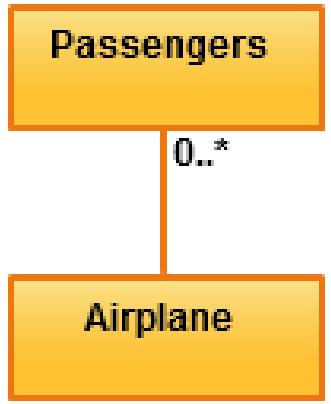
Association



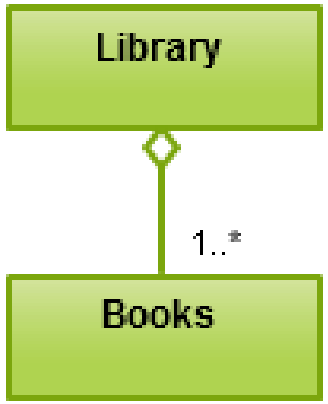
Directed Asscoation



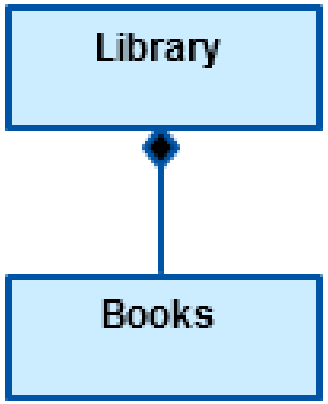
Reflexive Assciation



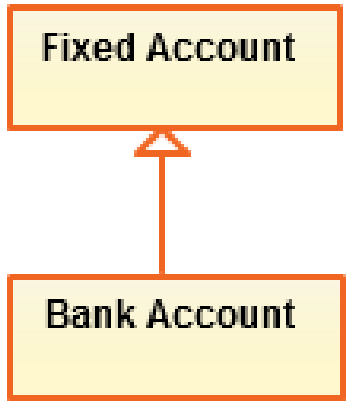
Multiplicity



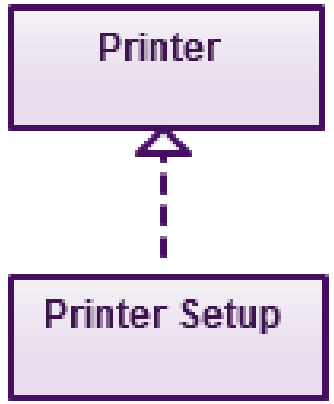
Aggregation



Composition



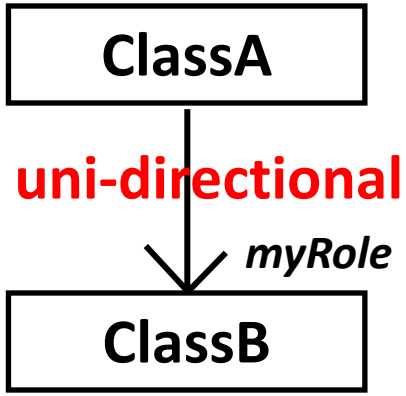
Inheritance



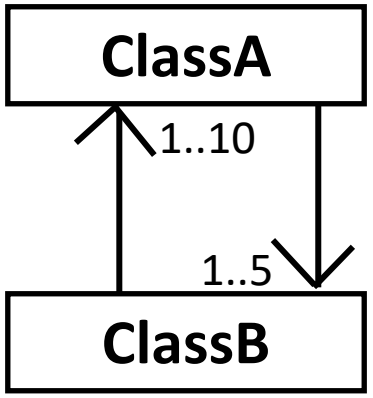
Realization

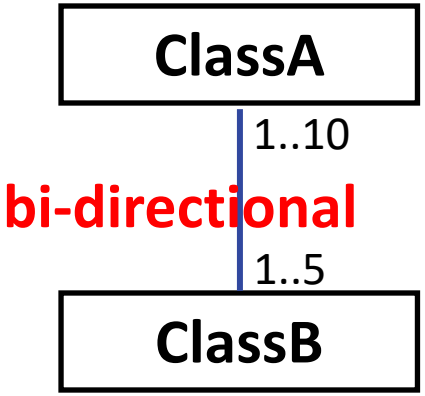
Association in Java Code

Association




```
class  ClassA  {
       ClassB  myRole ;  // attribute
       ……..
       …..
}
```






```
class  ClassA  {
       ClassB[ ] bObjs = new ClassB[ 5 ];  // attribute
       ……..
                    bObjs[0] = new ClassB(this) ;
……..
}
```

```
class  ClassB  {
   ClassA[ ] aObjs = new ClassA[ 10 ];  // attribute
       ……..
                public ClassB(ClassA a) { aObjs[0] =  a  ; …}
……..
}
```

Object composition

slido

Which of the following statement(s) describe(s) the relationship between a student and a teacher correctly?

ⓘ Start presenting to display the poll results on this slide.

Student

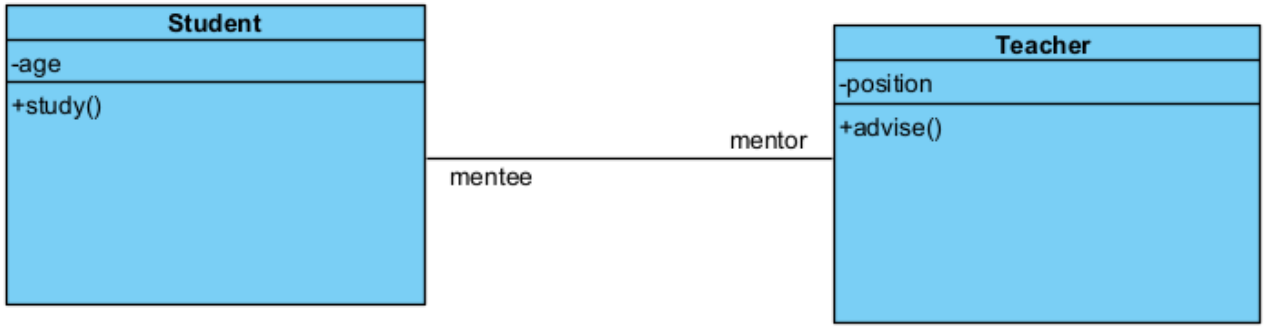
Teacher

A student knows the information of his teachers and A teacher knows the information of his students

A student knows the information of his teachers but a teacher does not need to know the information of his students

A teacher knows the information of his students but a student does not need to know the information of his teachers

Association



- Association between classes is bi-directional by default.
- A straight line

```
public class Student {

    Teacher mentor;
    private int age;

    public void study() {
        // TODO - implement Student.study
        throw new
UnsupportedOperationException();
    }

}
```

```
public class Teacher {

    Student mentee;
    private int position;

    public void advise() {
        // TODO - implement Teacher.advise
        throw new
UnsupportedOperationException();
    }

}
```

slido

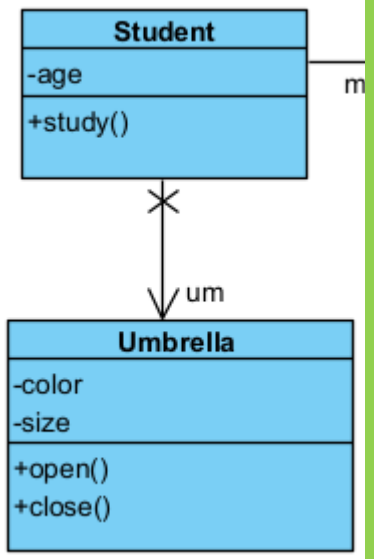
Which of the following statement(s) describe(s) the relationship between a student and his umbrella correctly?

ⓘ Start presenting to display the poll results on this slide.

Student

Umbrella

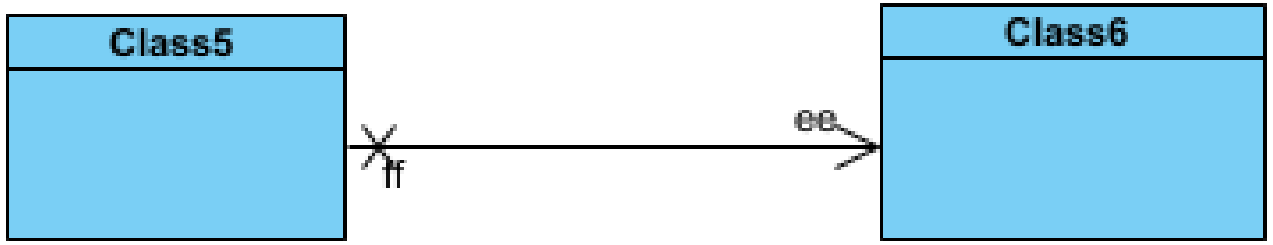
A student owns an umbrella
A student knows the information of his umbrella
An umbrella owns a student
An umbrella knows the information of his owner, who is a student




```
public class Student {

	Teacher mentor;
	private int age;
	Umbrella um;

	public void study() {
		// TODO - implement
Student.study
		throw new
UnsupportedOperationException();
	}

}
```

```
public class Umbrella {

	private int color;
	private int size;

	public void open() {
		// TODO - implement
Umbrella.open
		throw new
UnsupportedOperationException();
	}

	public void close() {
		// TODO - implement
Umbrella.close
		throw new
UnsupportedOperationException();
	}
}
```

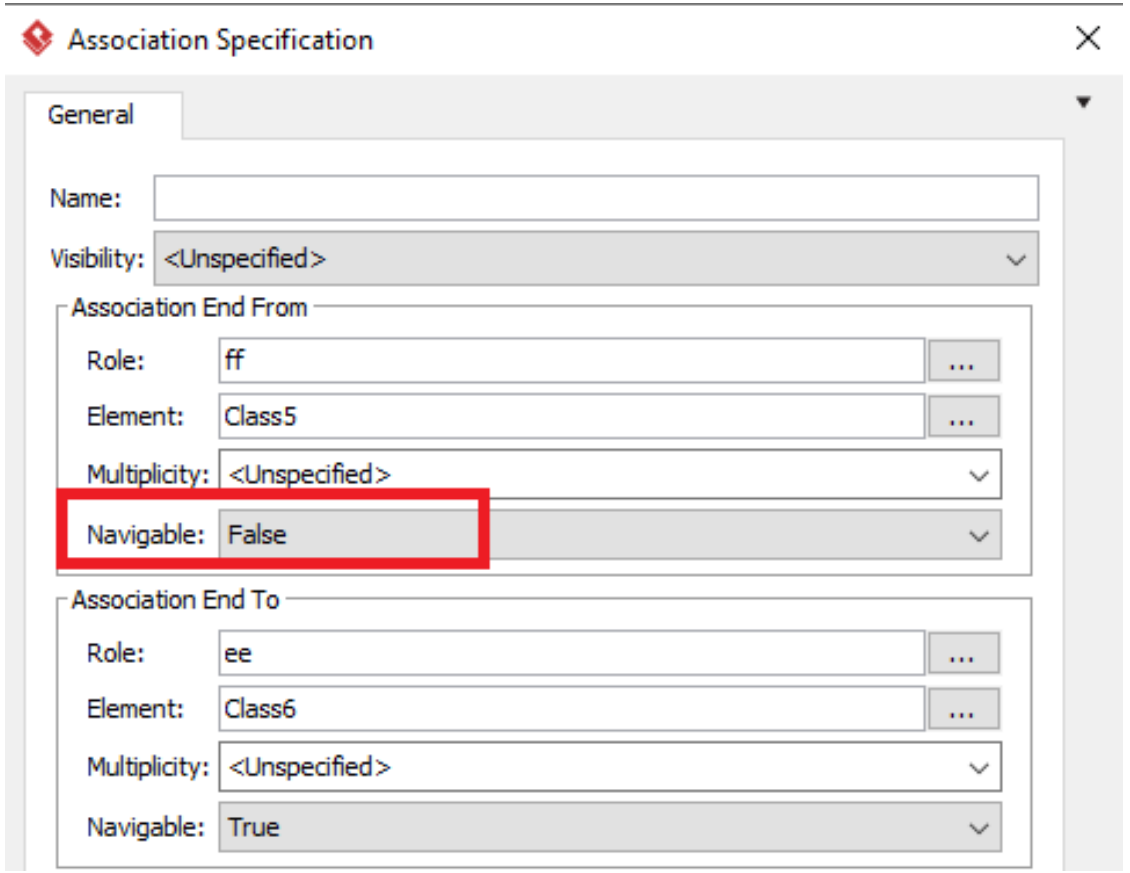
Visual paradigm can only recognize the uni-directional association in the following way. Explicitly close one end by setting Navigable to be false.



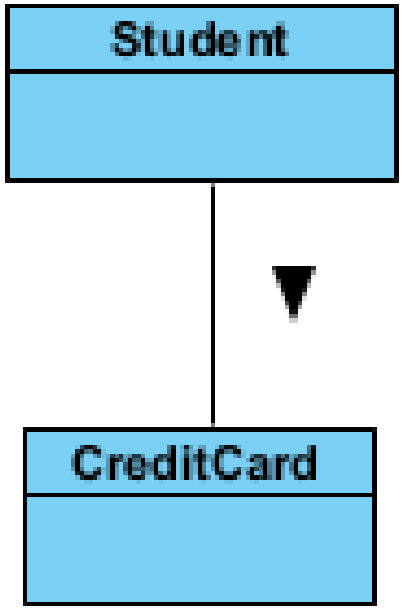
Auto generated code:

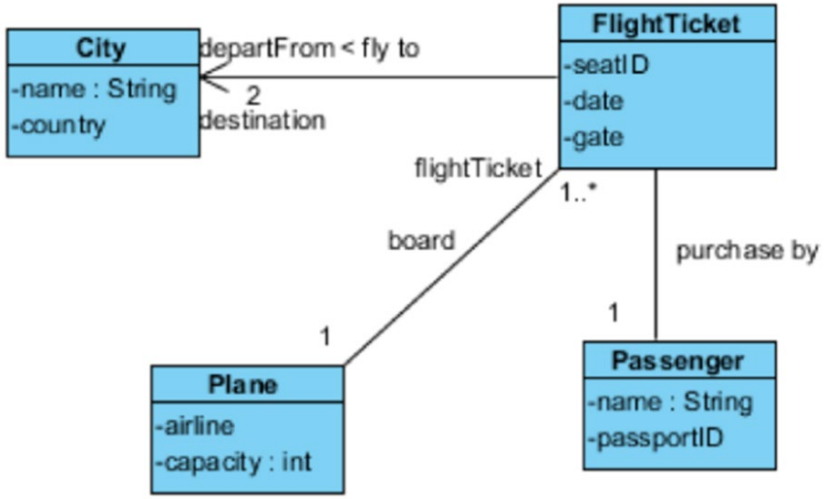
```
public class Class5 {

        Class6 ee;

}
```

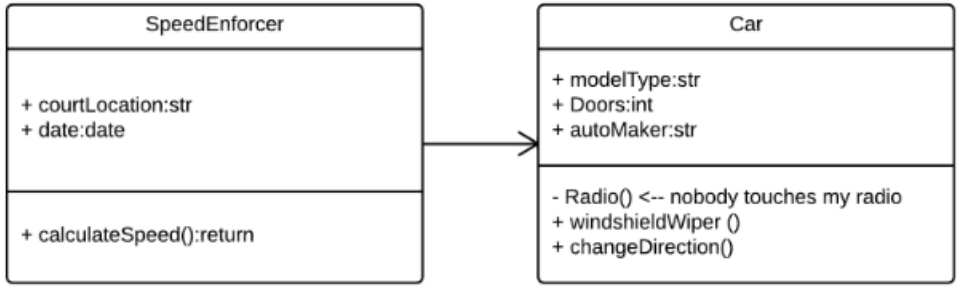
```
public class Class6 {
}
```

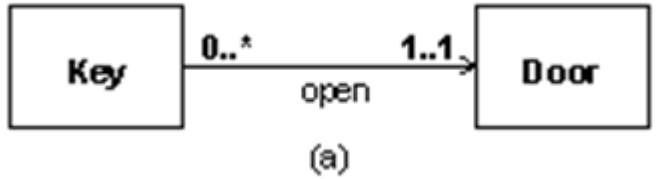


Directed Association （Uni-directional Association） in different UML tools (all are accepted in assignment class diagram drawing)

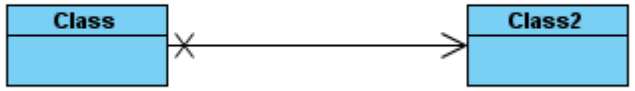


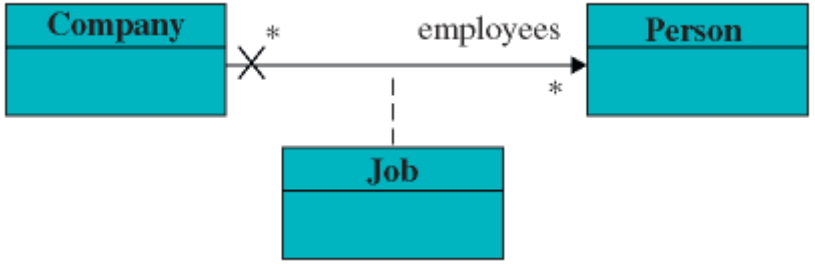






unidirectional

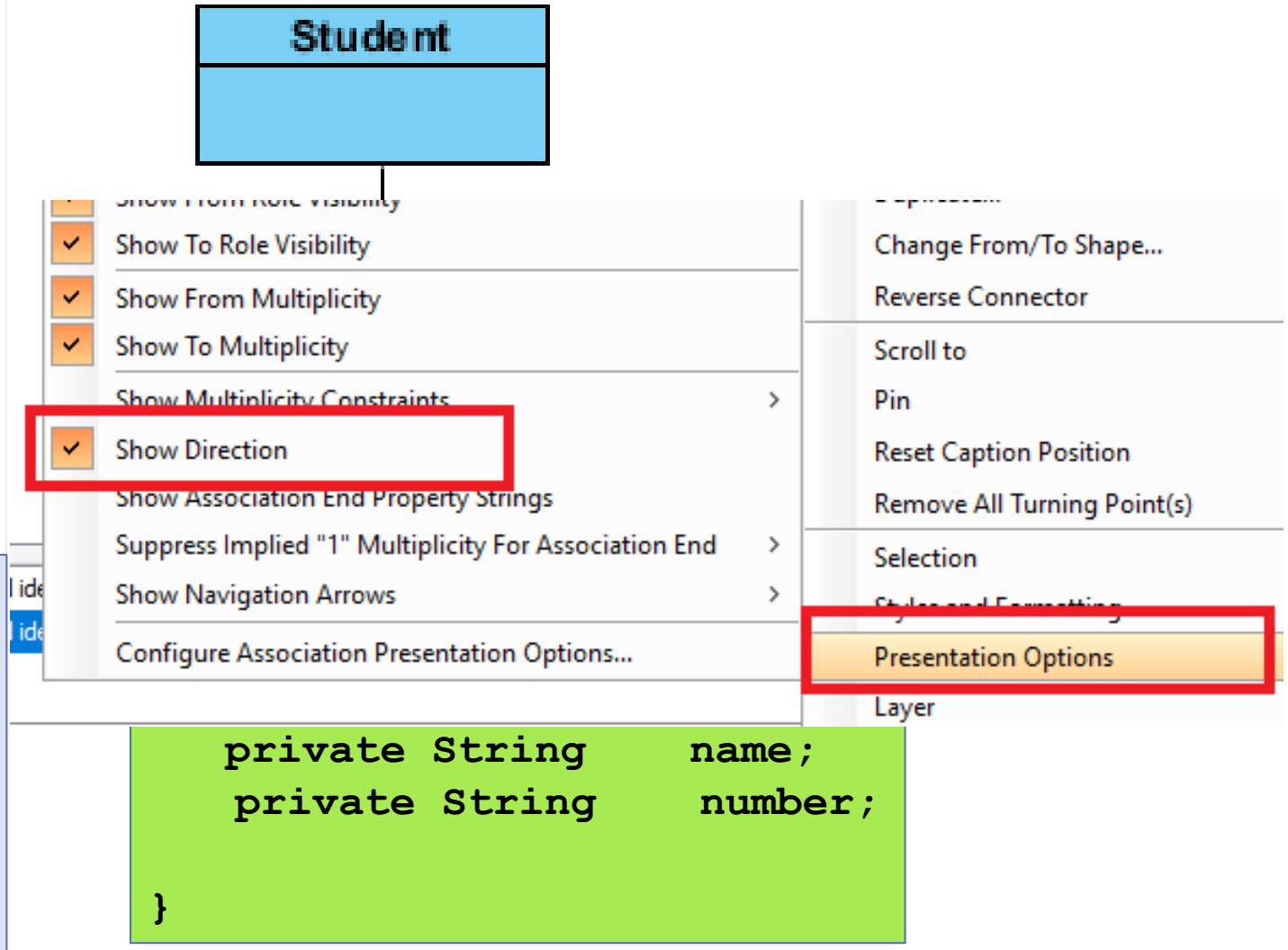




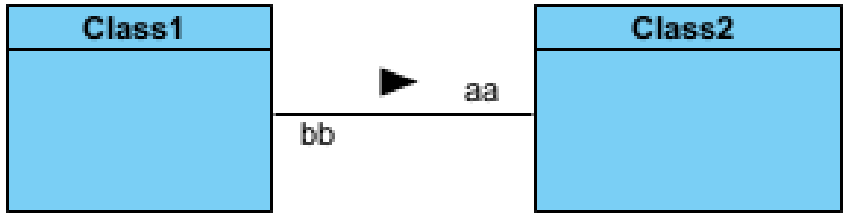
Directed Association （Uni-directional Association）

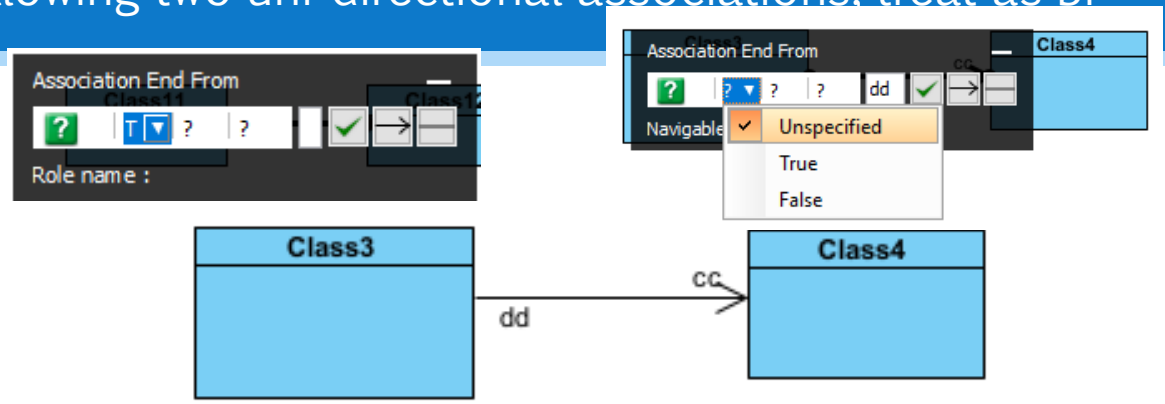
You can define the flow of the association by using a directed association. The arrowhead identifies the container-contained relationship.

```
public class Student {
    private int     studentId;
    private char    gender;
    private double height;
    private int     age;
    private CreditCard[] cards;
…..
}
```



Human being knows clearly that the following two indicate uni-directional associations. However, Visual paradigm code auto generator cannot recognize the following two uni-directional associations, treat as bi-directional association in auto coding.





Auto generated code:

```
public class Class1 {

        Class2 aa;

}
```

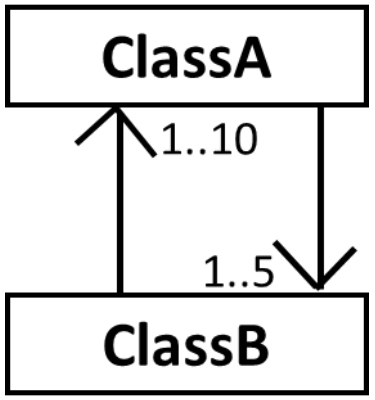
```
public class Class2 {

        Class1 bb;

}
```

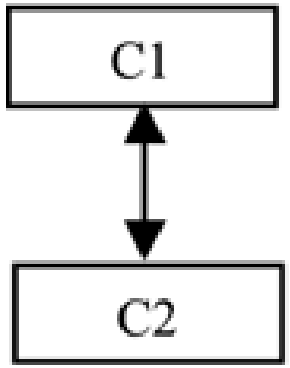
Auto generated code:

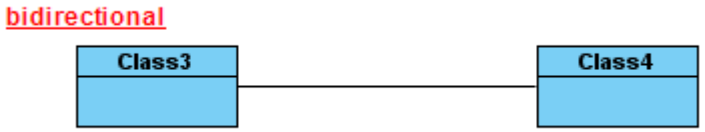
```
public class Class3 {

        Class4 cc;

}
```

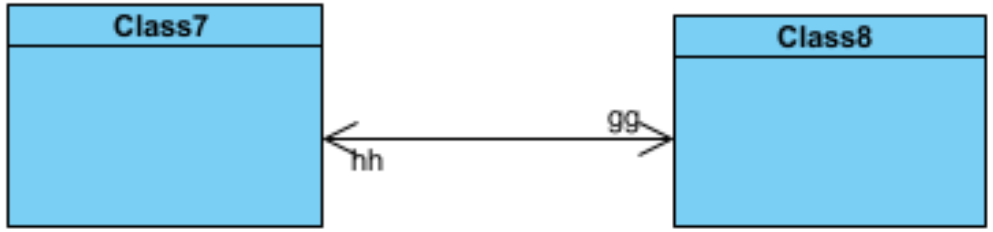
```
public class Class4 {

        Class3 dd;

}
```

bi-directional Association in different UML tools (all are accepted in exam class diagram drawing)

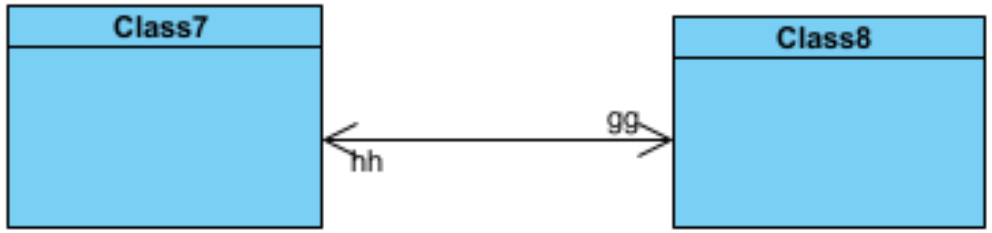


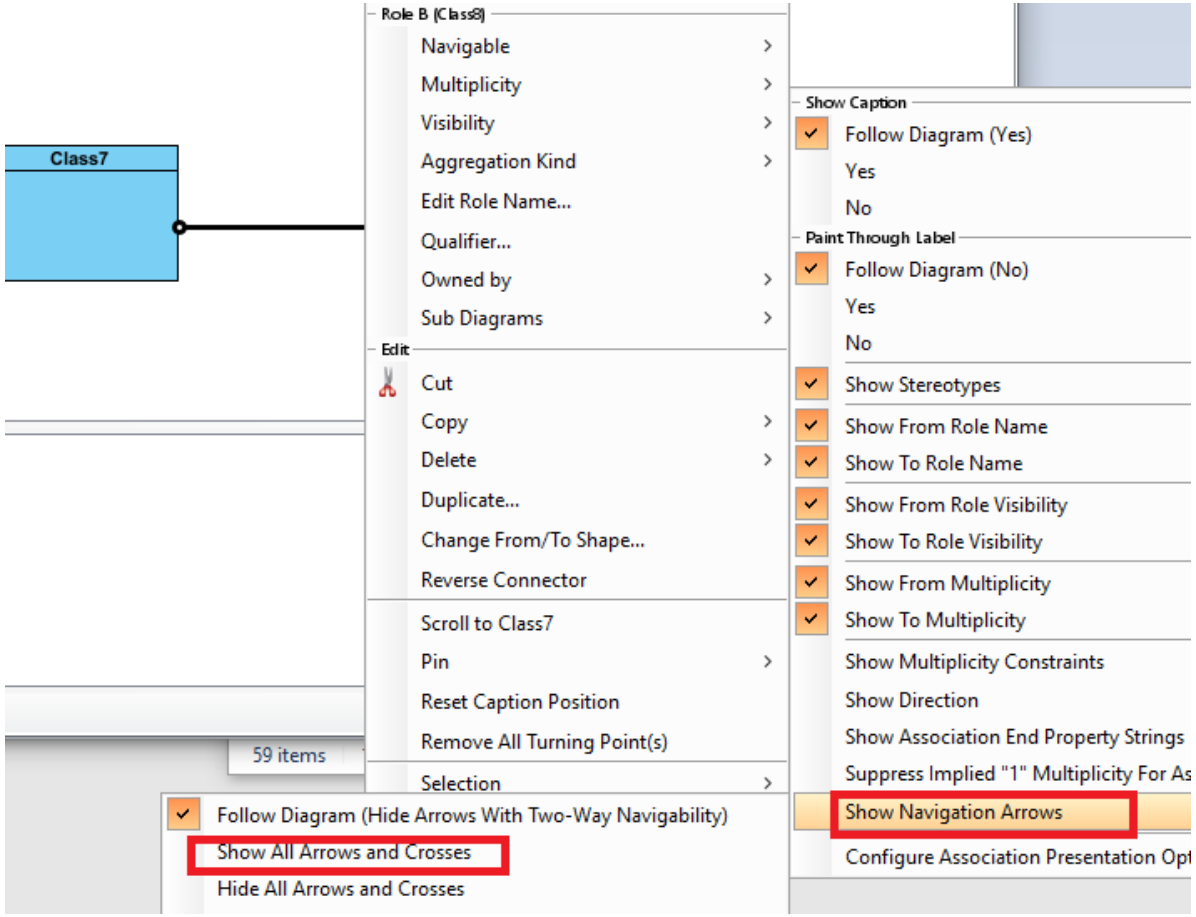




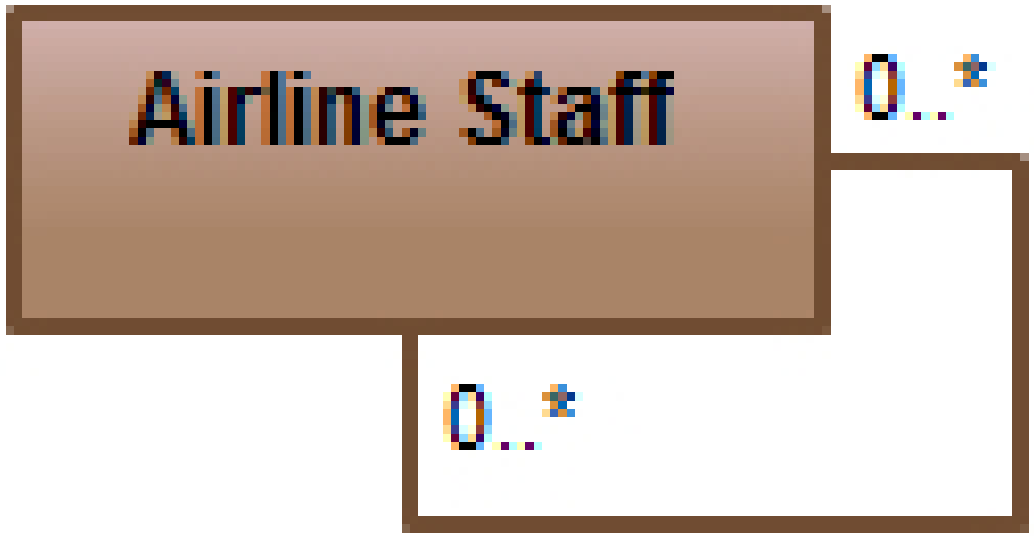


The default bi-directional association is a straight line without arrows. How to draw bi-directional association with explicit arrows in Visual paradigm?





Reflexive Association



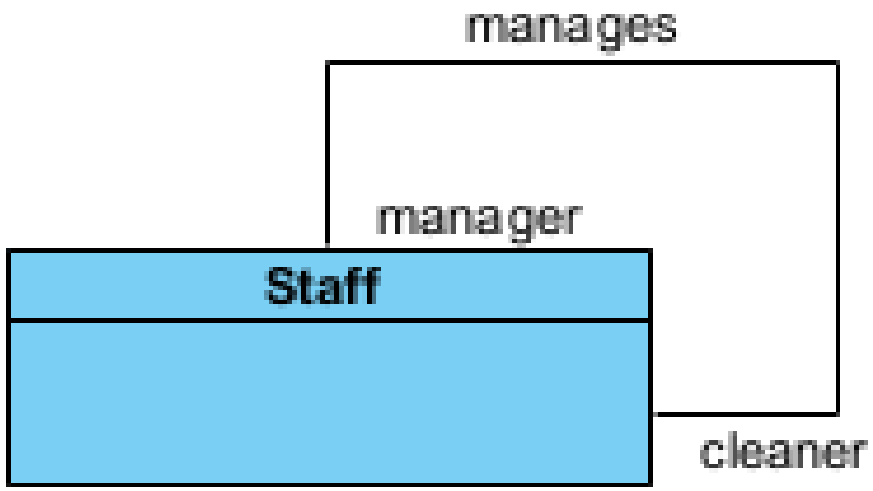
No separate symbol. However, the relation will point back at the same class.

This occurs when a class may have multiple functions or responsibilities. For example, a staff member working in an airport may be a pilot, aviation engineer, a ticket dispatcher, a guard, or a maintenance crew member. If the maintenance crew member is managed by the aviation engineer there could be a managed by relationship in two instances of the same class.

Reflexive Association

```
public class Staff{
        Staff manager;
        Staff cleaner;
}
```

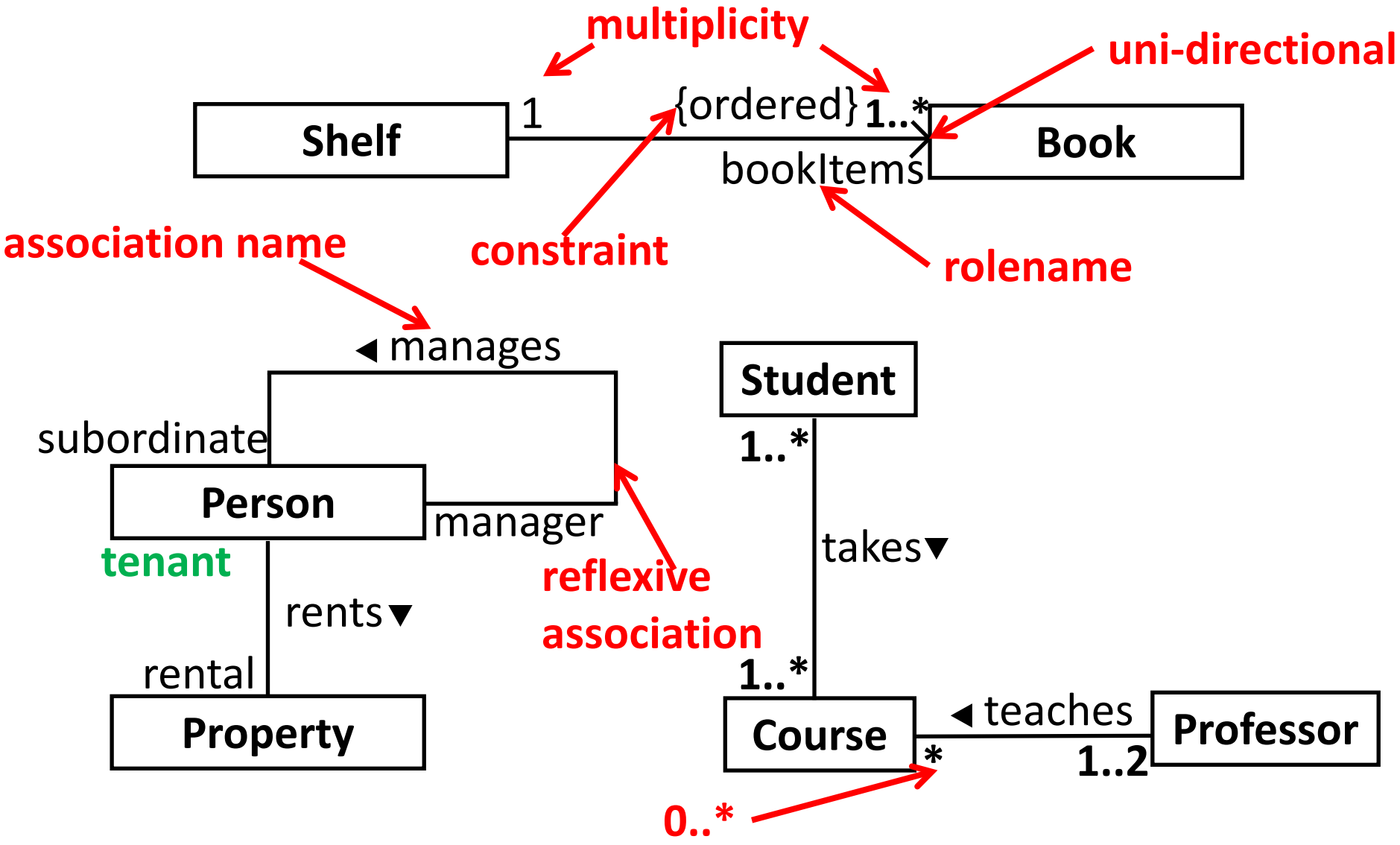
A manager, who is a staff, manages a cleaner, who is a staff as well.



Tutorial 6 Q1

- Given the following set of classes, draw a Class Diagram to show the appropriate relationship between them. Add multiplicity, role name and association name, if necessary:
  - Library, LibraryResource, Book, AudioVisual, Magazine
  - Driver, Car, Wheel, Engine
  - Plane, City, Passenger, FlightTicket
  - Company, Person, Department, Job
  - Product, Inventory, ItemStock, Catalog, Order, OrderLineItem, Manufacturer

Class Diagram - Association



Topic 3: Association name

slido

Please give an appropriate verb to link the following two classes.

ⓘ Start presenting to display the poll results on this slide.

Which of the following verb is appropriate to link the following two classes?

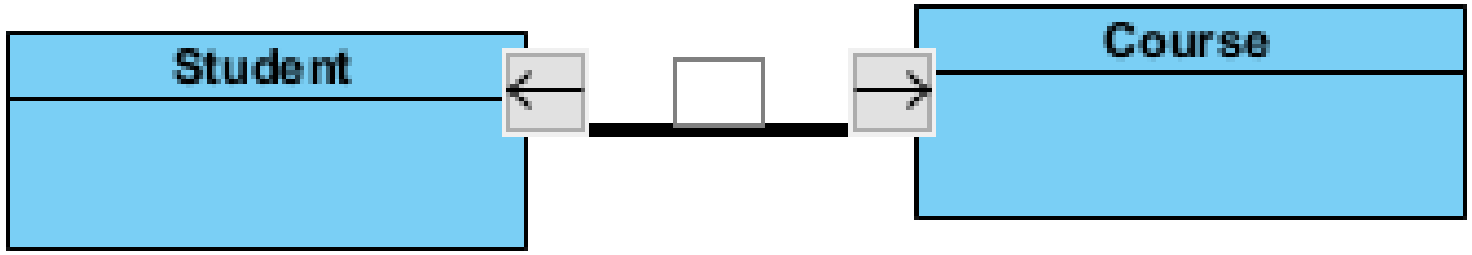
A. teaches

B. paints

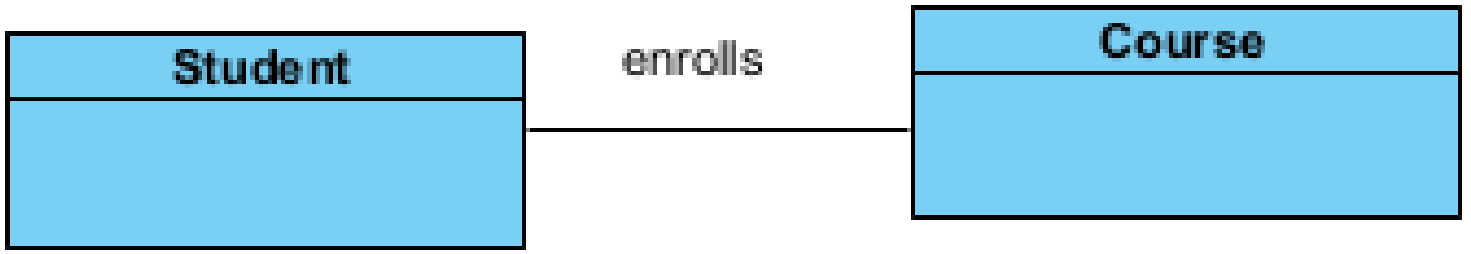
✓ C. enrols

D. drinks

Association name



Association names should reflect the purpose of the relationship and be a verb phrase.



Name of the association can be shown somewhere near the middle of the association line but not too close to any of the ends of the line.

Topic 4: Association Role Names

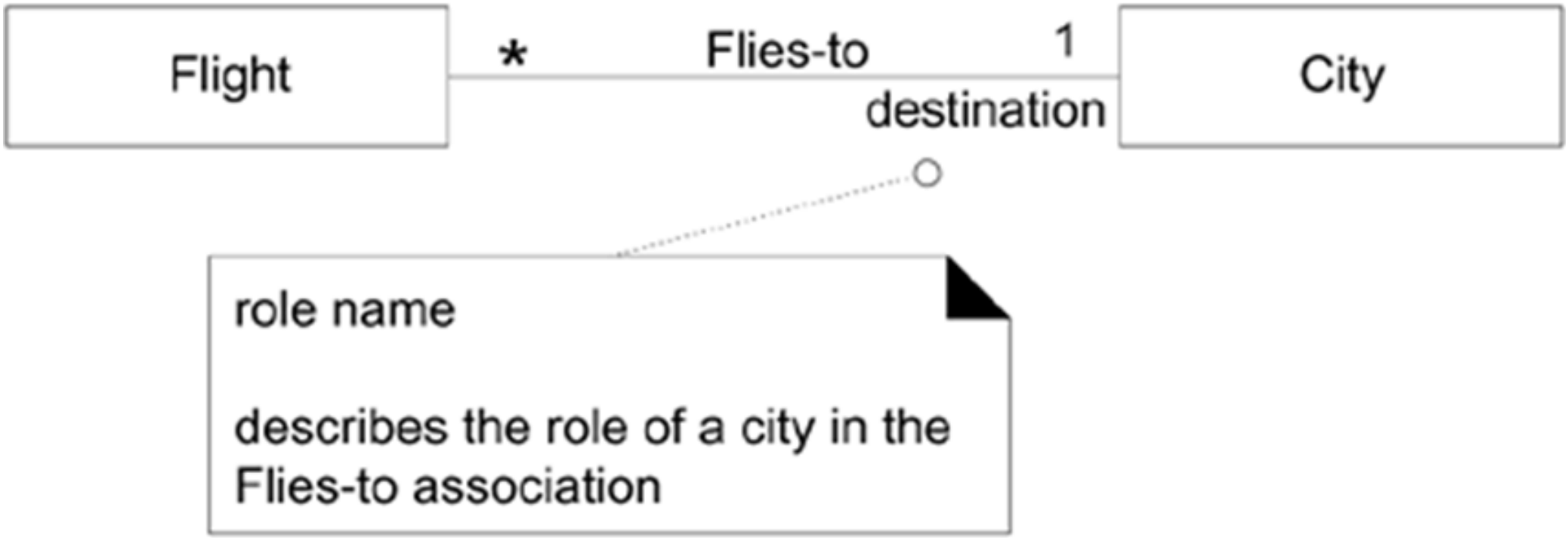
Each end of an association is a role, which has various properties, such as:

- name
- multiplicity

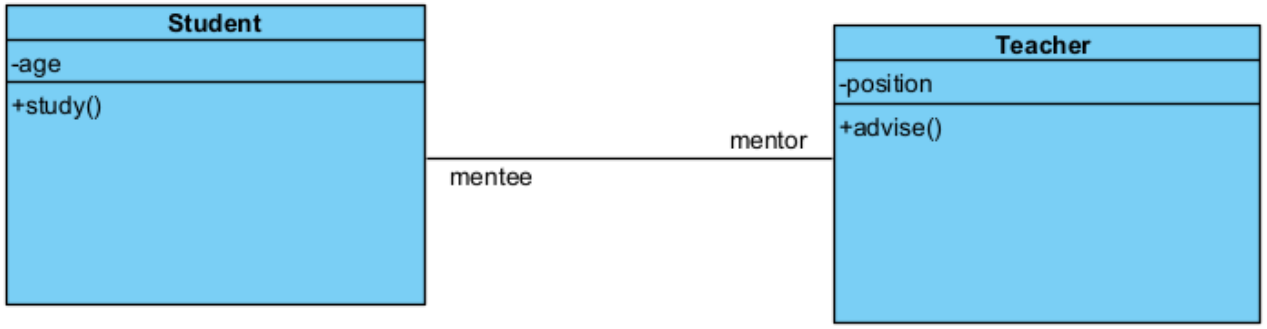
A role name identifies an end of an association and ideally describes the role played by objects in the association.

Roles: A role name explains how an object participates in the relationship.

Association ***Wrote*** *between* ***Professor*** *and* ***Book*** *with association ends* ***author*** *and* ***textbook****.*


Flight
*
Flies-to
1
destination
City
role name
describes the role of a city in the Flies-to association

Association

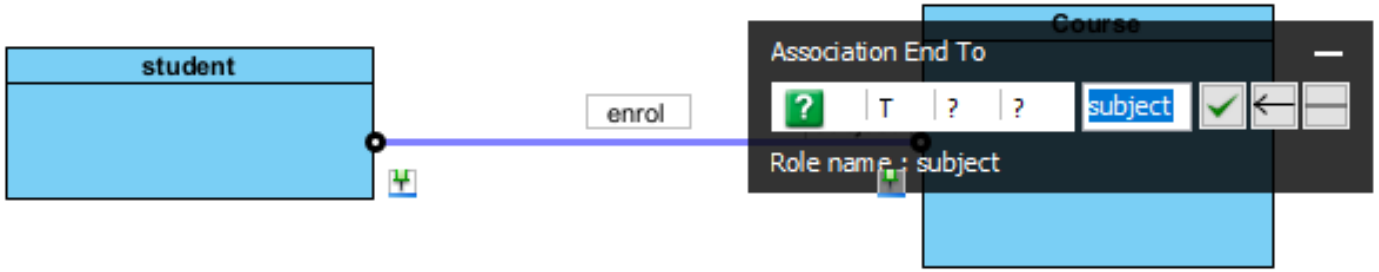


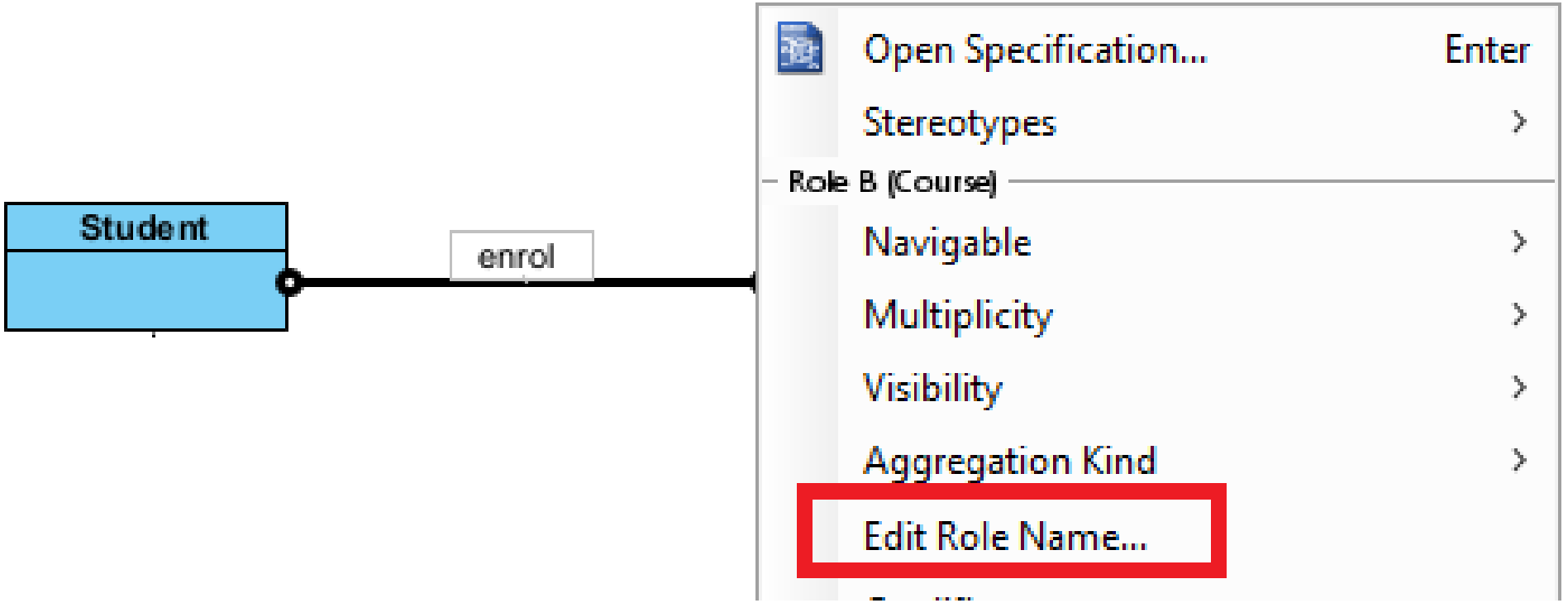
- Association between classes is bi-directional by default.
- A straight line

```
public class Student {

    Teacher mentor;
    private int age;

    public void study() {
        // TODO - implement Student.study
        throw new
UnsupportedOperationException();
    }

}
```

```
public class Teacher {

    Student mentee;
    private int position;

    public void advise() {
        // TODO - implement Teacher.advise
        throw new
UnsupportedOperationException();
    }

}
```

Visual Paradigm-edit role name





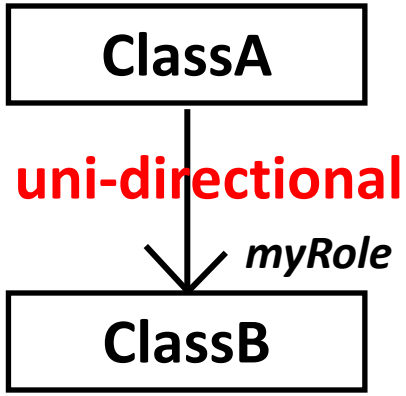
Topic 5: Multiplicity

Multiplicity

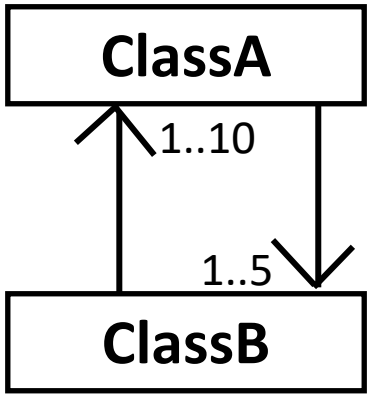
- **Multiplicity** is used to describe how many instances participate in **has-a** relationships, but it doesn’t apply to **is-a** (inheritance) or **use-a** (dependency) relationships.
- **Has-a Relationships** (Association, Aggregation, Composition):
- In these relationships, multiplicity specifies how many instances of one class are related to instances of another class. For example:
  - A **Library** (1) "has" many **Books** (*), or
  - A **Car** (1) "has" 4 **Wheels** (4).

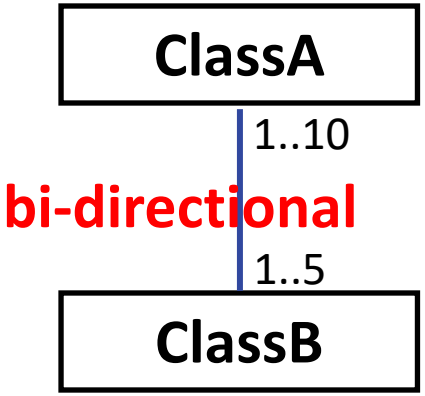
Association in Java Code

Association




```
class  ClassA  {
       ClassB  myRole ;  // attribute
       ……..
       …..
}
```






```
class  ClassA  {
       ClassB[ ] bObjs = new ClassB[ 5 ];  // attribute
       ……..
                    bObjs[0] = new ClassB(this) ;
……..
}
```

```
class  ClassB  {
   ClassA[ ] aObjs = new ClassA[ 10 ];  // attribute
       ……..
                public ClassB(ClassA a) { aObjs[0] =  a  ; …}
……..
}
```

Object composition

Multiplicities examples:

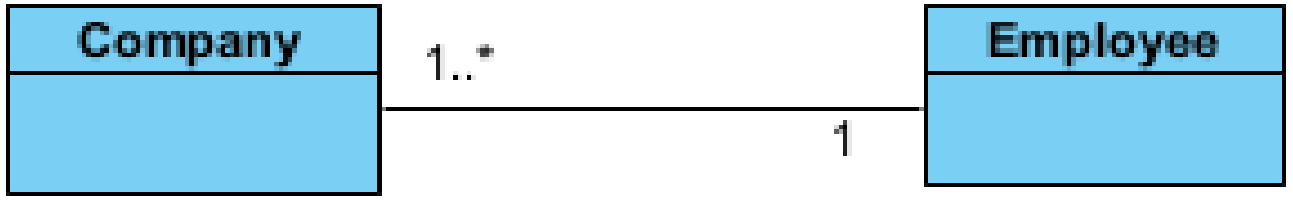
1	Exactly one, no more and no less
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many

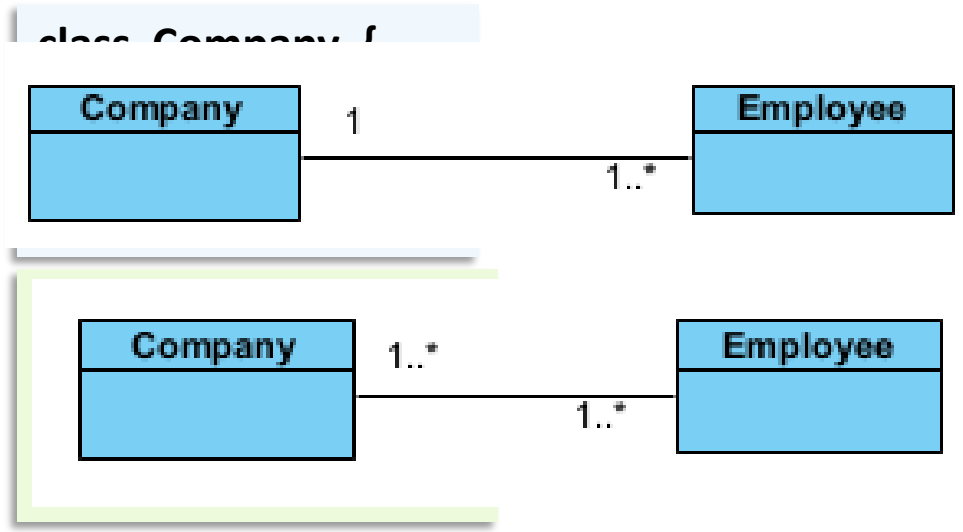
slido

Is the following multiplicity appropriate?

ⓘ Start presenting to display the poll results on this slide.

Is the following multiplicity appropriate?





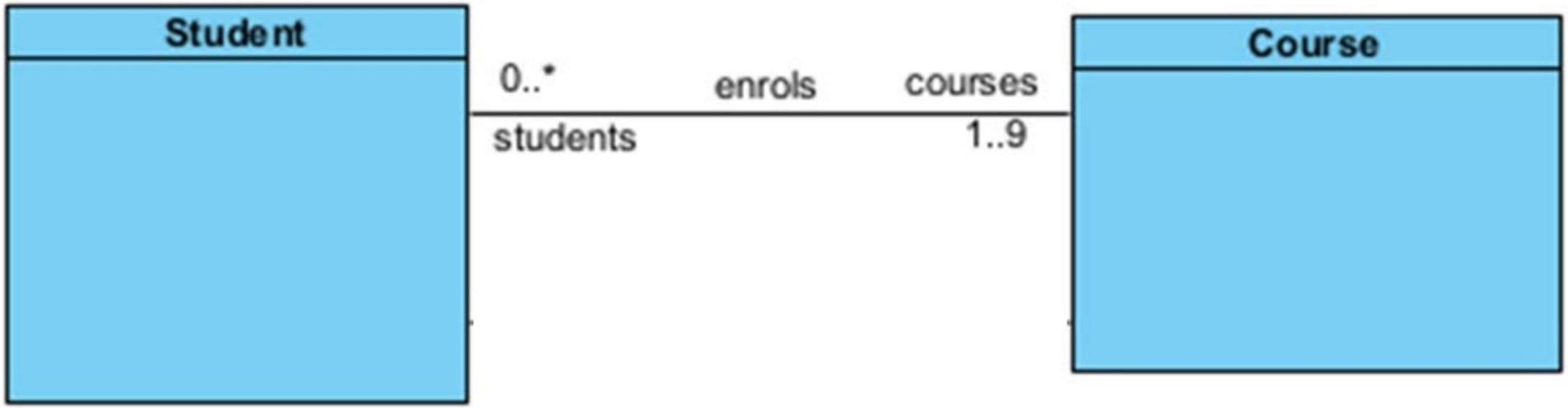
A. Yes

✓ B. No

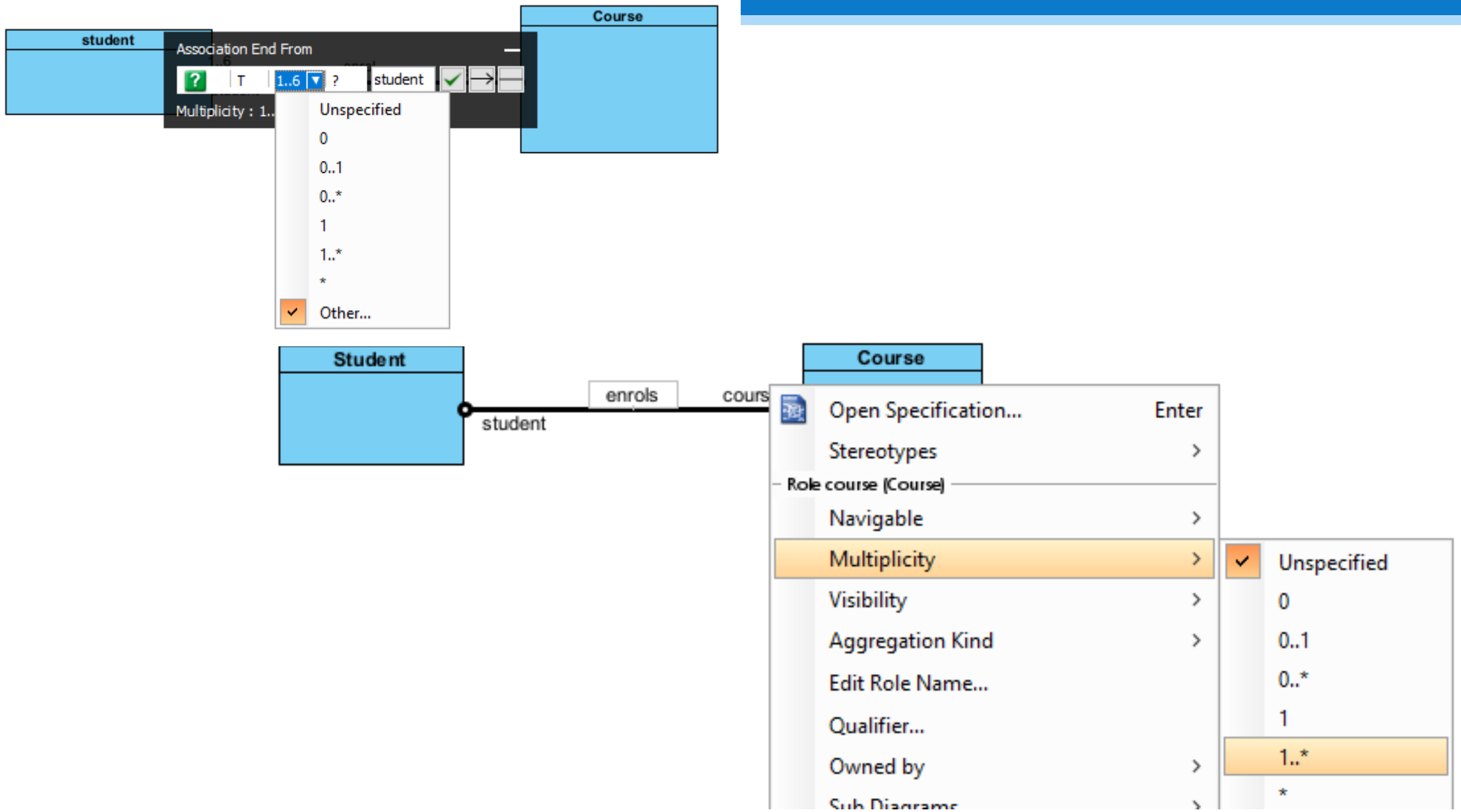
slido

What are the correct statements about the class diagram?

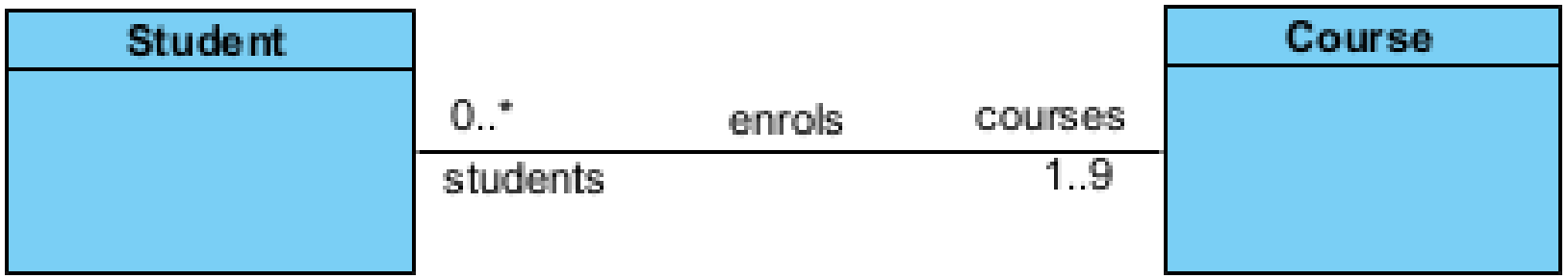
ⓘ Start presenting to display the poll results on this slide.


Student
0..*
enrols
courses
students
1..9
Course

Demo-multiplicity



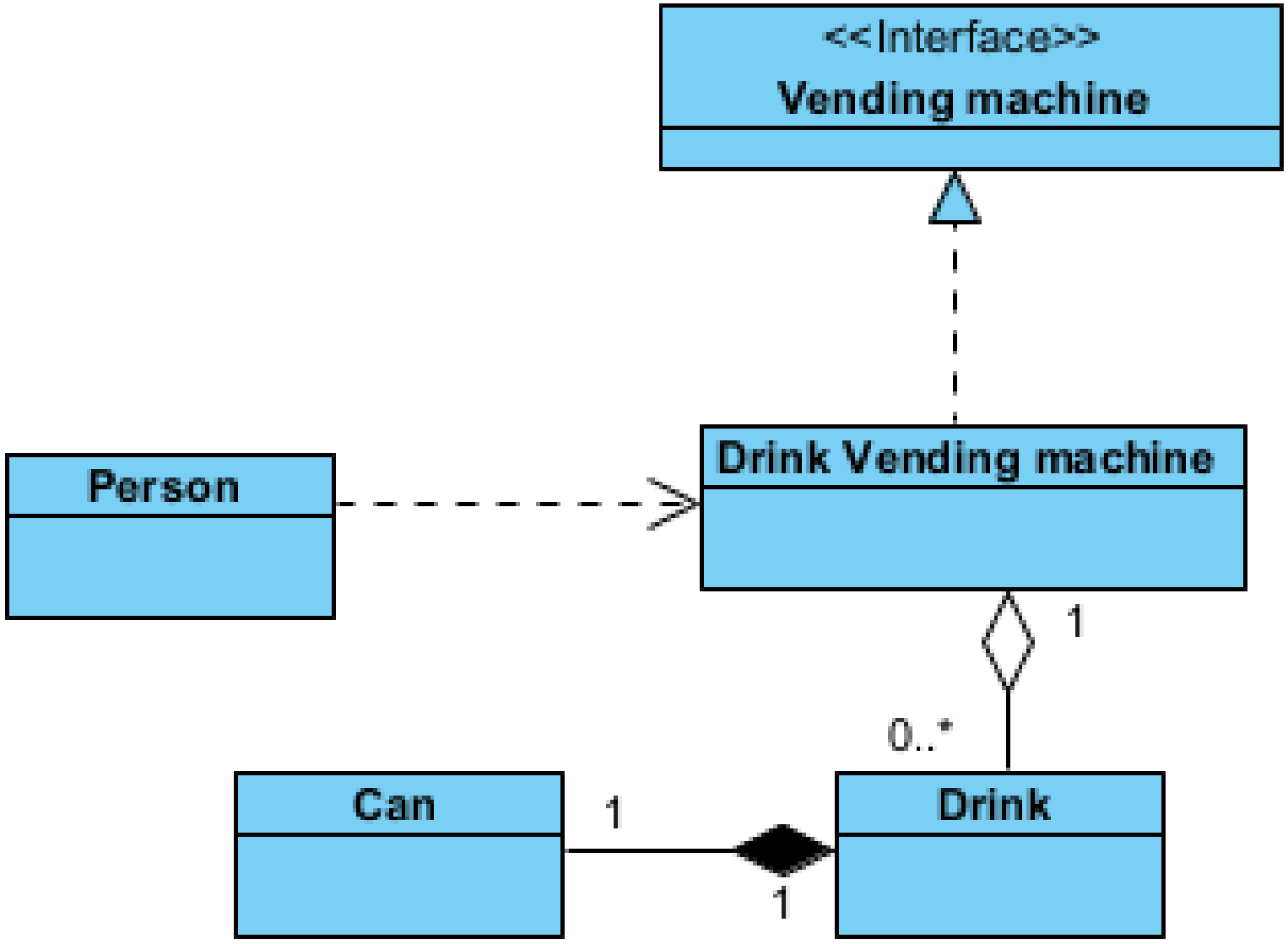
Demo




```
import java.util.*;

public class Student {

	Collection<Course> courses;

}
```

```
import java.util.*;

public class Course {

	Collection<Student> students;

}
```

Multiplication only work on has-a category-association



Topic 6: Constraints

In **UML class diagrams**, constraints are usually written as notes or comments alongside the relationship, specifying conditions like "must", "should", or limits on the relationship.
Constraints typically apply to **has-a** relationships (association, aggregation, composition) to enforce rules about how objects are related. They may also apply to **use-a** relationships (dependency), but they are not typically used in **is-a** (inheritance) relationships.

Constraints in a class-diagram intend to express general statements about the class and its instances.

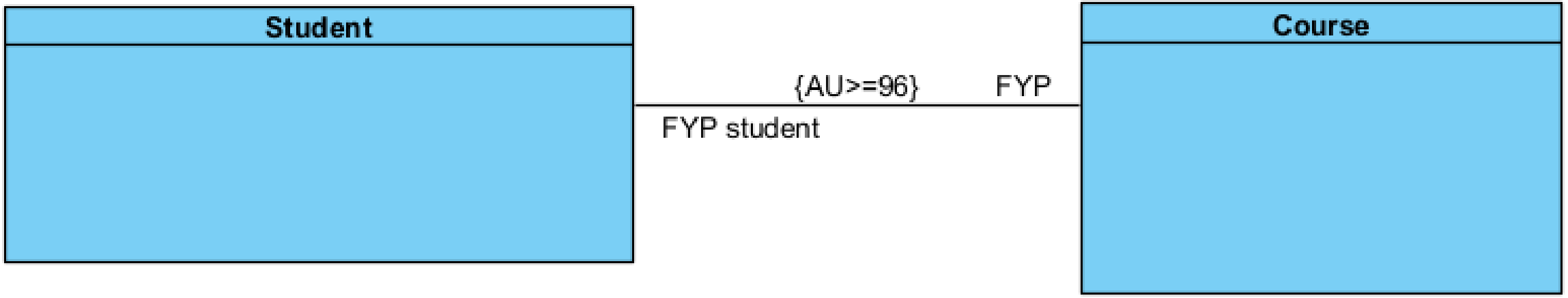
A project which is more than 1 million dollar will require a a project director.

Maximally, a staff can be a project director for two projects.

slido

What are the correct statements about the class diagram?

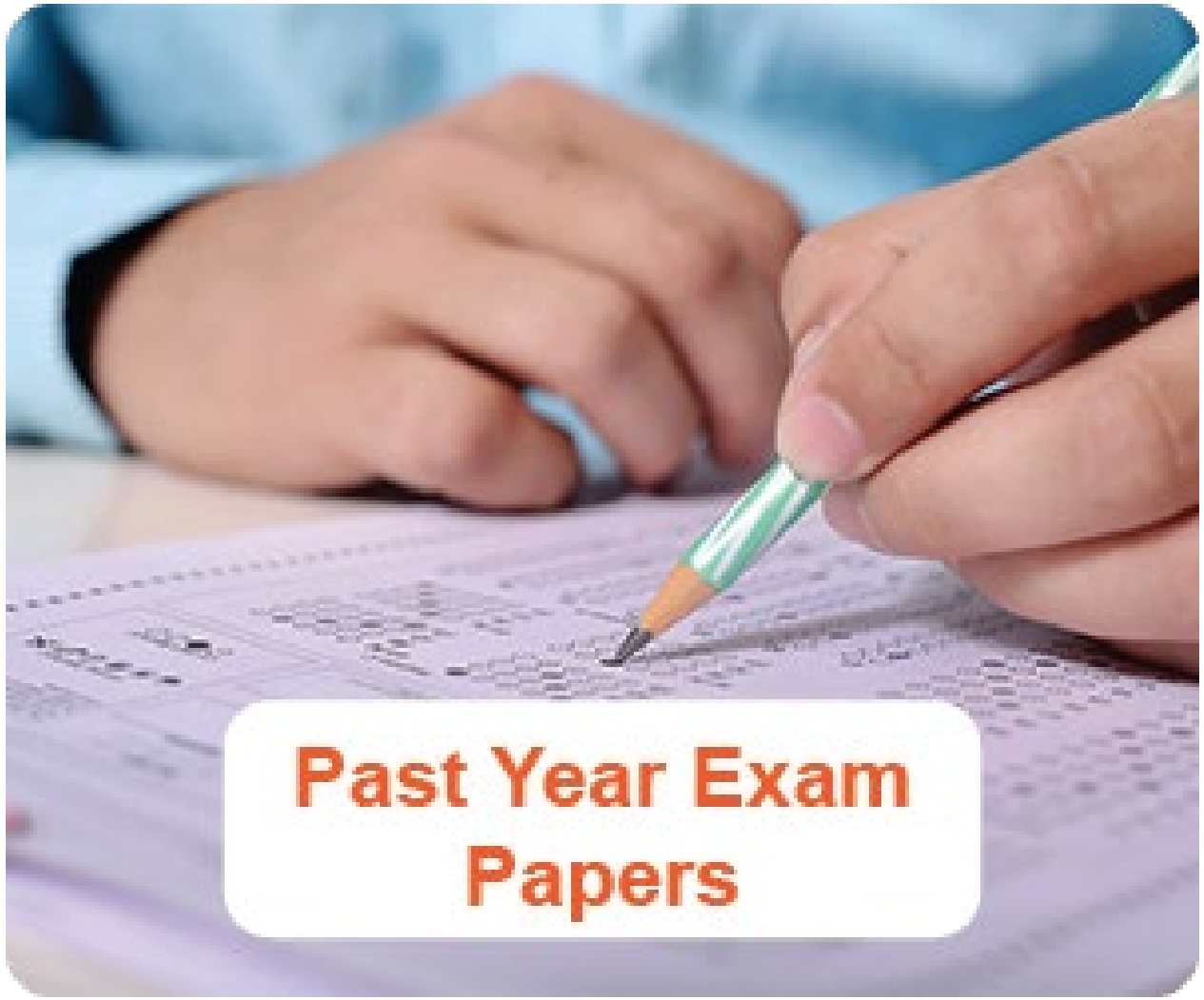
ⓘ Start presenting to display the poll results on this slide.


Student
{AU>=96}
FYP
FYP student
Course

Constraints in Use-a

- **Use-a Relationships (Dependency)**
- Constraints can be applied to **use-a** relationships to define when and how a dependency is invoked.
- **Example**: "A payment processing system must use a payment gateway, but only if the transaction amount exceeds a certain threshold."

PYP revision



2020 Nov Q3a

Study the following description of a Work Tracking Application (WTA):

- WTA is a system that maintains a centralized repository of all projects and tasks assigned to staff in an organisation. It contains the project and task details, including the tracking of issues. WTA allows the easy access of relevant information on the staff and projects as well as tracking a project's progress and staff's utilisation rate. A staff can play different roles in projects.
- A project has an identification number, a description, a project manager, the project awarded price and the customer. A project which is more than 1 million dollar will require also a project director. A project is broken down into multiple tasks which are assigned to the same or different staff. Each task will have a project leader, its team members, a task identification, task description and an expected date of completion. Project issues are tracked for every task in the project by assigning an identification number and providing details like title, detailed description, status, reported date and last updated date. A project is funded externally by a customer which the system will store the company name, company registration number, contact person and number.

- WTA also stores the staff details like identification number, name, assigned projects and tasks, skillsets and man-hour rate. Staff will be assigned to ONE to TWO projects but multiple tasks within the same project depending on their skillsets. Skillsets like certification name, the level, awarded date and date of expiry are stored. Every month, each staff is required to clock his/her monthly utilisation details by recording the time spent on the assigned projects and tasks. Details such as the month and year, period of work (from date and to date) and type of work performed. The type of work can be administrative, project, on leave, training and mentoring.

You are tasked to identify the entity classes needed to build the application based on the description above.

Show your design in a Class Diagram. Your Class Diagram should show clearly the relationships between classes, enumeration, relevant attributes (at least TWO), logical multiplicities, meaningful role names, association names and constraint(s), if any. You need not show the class methods.

Steps:

1. Entity classes: Highlight the keywords of classes (Nouns)
   - Those Nouns which have been repeated for a few times
   - Those Nouns which have detailed description/definition
2. Relationship: Highlight the keywords of relationships (Verbs)
   - The verbs indicate “is part of” relationship
   - The verbs indicate “is a special” relationship
   - The verbs indicate normal association relationship
3. Attribute: Those nouns in description/definition of Entity classes
4. Role name: Those nouns which are instances of some entity classes in description/definition of Entity classes
5. Multiplicity: Highlight the words related to quantity
   - a
   - One to two
   - s. e.g., team members

slido

What are the nouns, which have been repeated for a few times in following paragraph?

ⓘ Start presenting to display the poll results on this slide.

What are the nouns, which have been repeated for a few times in following paragraph?

WTA is a system that maintains a centralized repository of all projects and tasks assigned to staff in an organisation. It contains the project and task details, including the tracking of issues. WTA allows the easy access of relevant information on the staff and projects as well as tracking a project's progress and staff's utilisation rate. A staff can play different roles in projects.

A. System, repository, project, staff
B. Project, staff
✓ C. Project, task, staff
D. WTA, Project, task, staff

slido

What are the nouns, which have detailed description/definition in following paragraph, but not in the previous list?

ⓘ Start presenting to display the poll results on this slide.

What are the nouns, which have detailed description/definition in following paragraph, but not in the previous list?

A project has an identification number, a description, a project manager, the project awarded price and the customer. A project which is more than 1 million dollar will require also a project director. A project is broken down into multiple tasks which are assigned to the same or different staff. Each task will have a project leader, its team members, a task identification, task description and an expected date of completion. Project issues are tracked for every task in the project by assigning an identification number and providing details like title, detailed description, status, reported date and last updated date. A project is funded externally by a customer which the system will store the company name, company registration number, contact person and number.

A. ProjectIssue, customer, description
✓ B. ProjectIssue, customer
C. ProjectIssue, customer, company
D. Project, ProjectIssue, customer

slido

What are the nouns, which have detailed description/definition in following paragraph, but not in the previous lists?

ⓘ Start presenting to display the poll results on this slide.

What are the nouns, which have detailed description/definition in following paragraph, but not in the previous lists?

WTA also stores the staff details like identification number, name, assigned projects and tasks, skillsets and man-hour rate. Staff will be assigned to ONE to TWO projects but multiple tasks within the same project depending on their skillsets. Skillsets like certification name, the level, awarded date and date of expiry are stored. Every month, each staff is required to clock his/her monthly utilisation details by recording the time spent on the assigned projects and tasks. Details such as the month and year, period of work (from date and to date) and type of work performed. The type of work can be administrative, project, on leave, training and mentoring.

✓ A. Skillsets, monthlyutilisation, typeofwork

B. identificationNumber, typeofwork

C. Skillsets, monthlyutilisation

3. (a) Study the following description of a Work Tracking Application (WTA):

WTA is a system that maintains a centralized repository of all projects and tasks assigned to staff in an organisation. It contains the project and task details, including the tracking of issues. WTA allows the easy access of relevant information on the staff and projects as well as tracking a project's progress and staff's utilisation rate. A staff can play different roles in project.

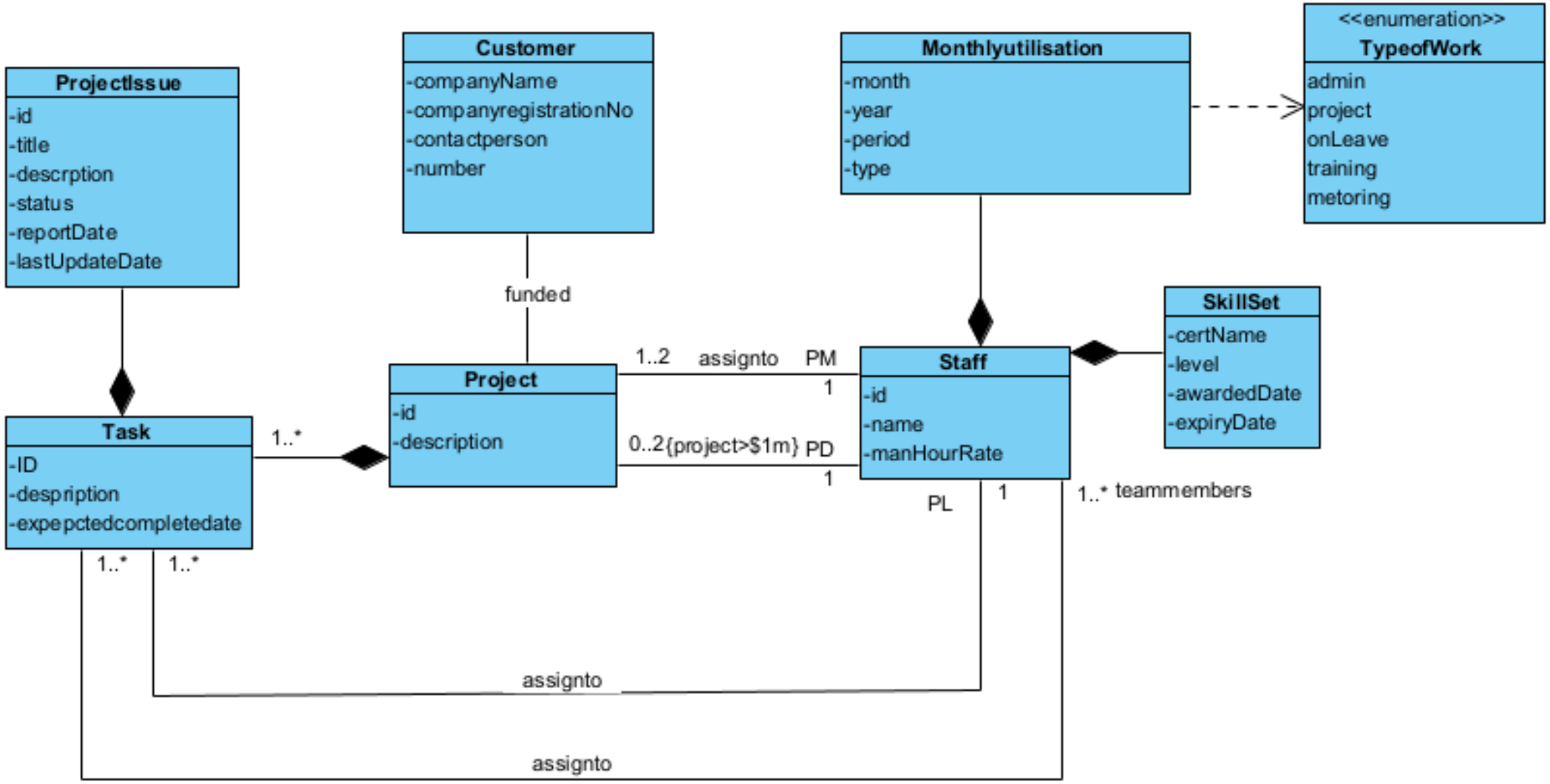
role

constraints

A project has an identification number, a description, a **project manager**, the project awarded price and the customer. A project which is more than 1 million dollar will require also a **project director**. A project is broken down into multiple tasks which are assigned to the same or different staff. Each task will have a **project leader**, its **team members**, a task identification, task description and an expected date of completion. Project issues are tracked for every task in the project by assigning an identification number and providing details like title, detailed description, status, reported date and last updated date. A project is funded externally by a customer which the system will store the company name, company registration number, contact person and number.

WTA also stores the staff details like identification number, name, assigned projects and tasks, skillsets and man-hour rate. Staff will be assigned to ONE to TWO projects but multiple tasks within the same project depending on their skillsets. Skillsets like certification name, the level, awarded date and date of expiry are stored. Every month, each staff is required to clock his/her monthly utilisation details by recording the time spent on the assigned projects and tasks. Details such as the month and year, period of work (from date and to date) and type of work performed. The type of work can be administrative, project, on leave, training and mentoring.

enumeration



Correct classes : **5 marks** (esp Staff, Project, Task, Issue, Utilisation)
WorkType as enum : 1 mark
Constraints for project cost > 1 mil to have Proj Dir : 1 mark
Correct use of association, composition : **2 marks**
Attributes **: 2 marks** (no private -1)
Correct logical Multiplicity : **2 marks** (0..*/1..*) -- show **0..2**
Association name : 1 mark

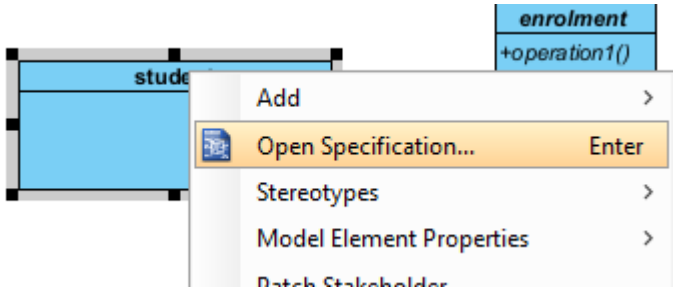
```
public class Project {

	Staff PD;
	Staff PM;
	private int id;
	private int description;

}
```

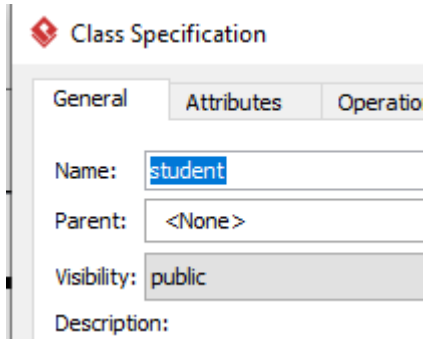
```
public class Staff {

	private int id;
	private int name;
	private int manHourRate;

}
```

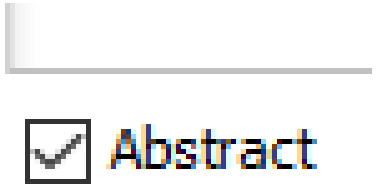
```
import java.util.*;

public class Task {

	Staff PL;
	Collection<Staff> teammembers;
	private int ID;
	private int description;
	private int expepctedcompletedate;

}
```

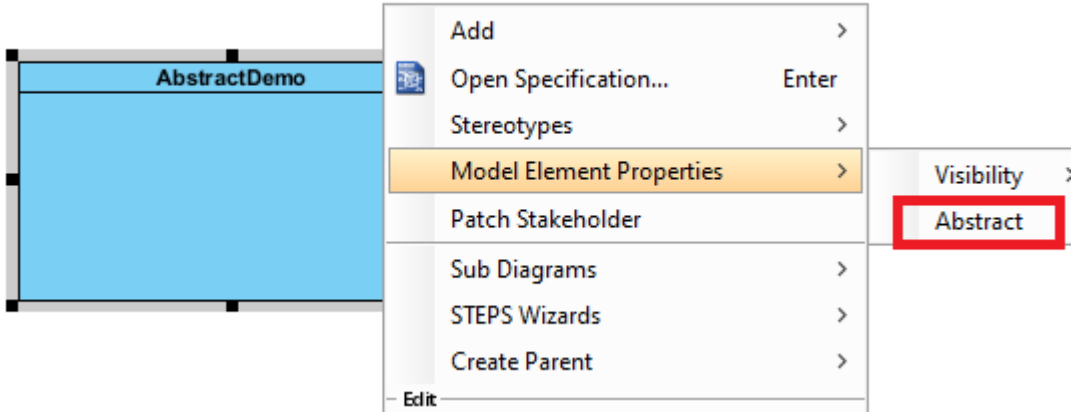
```
public class ProjectIssue {

	private int id;
	private int title;
	private int description;
	private int status;
	private int reportDate;
	private int lastUpdateDate;

}
```

Topic 7: Abstract class



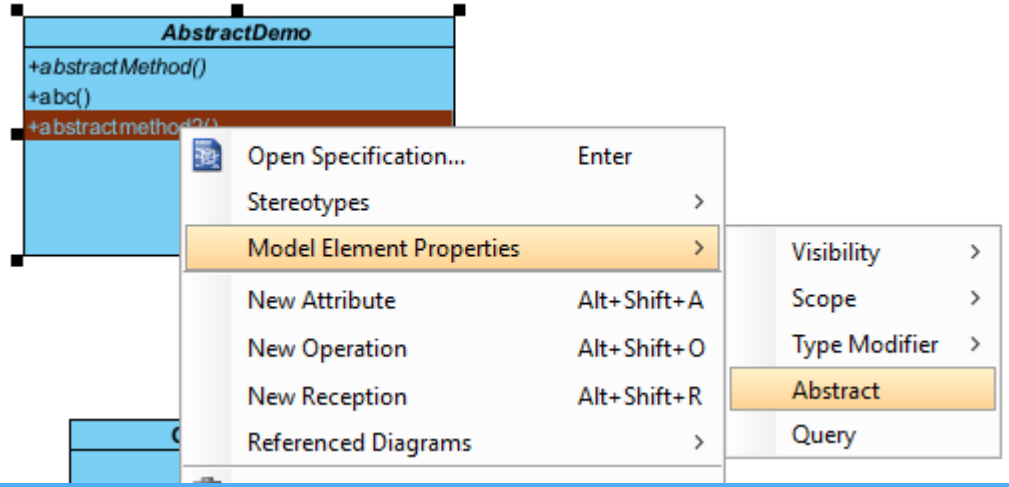


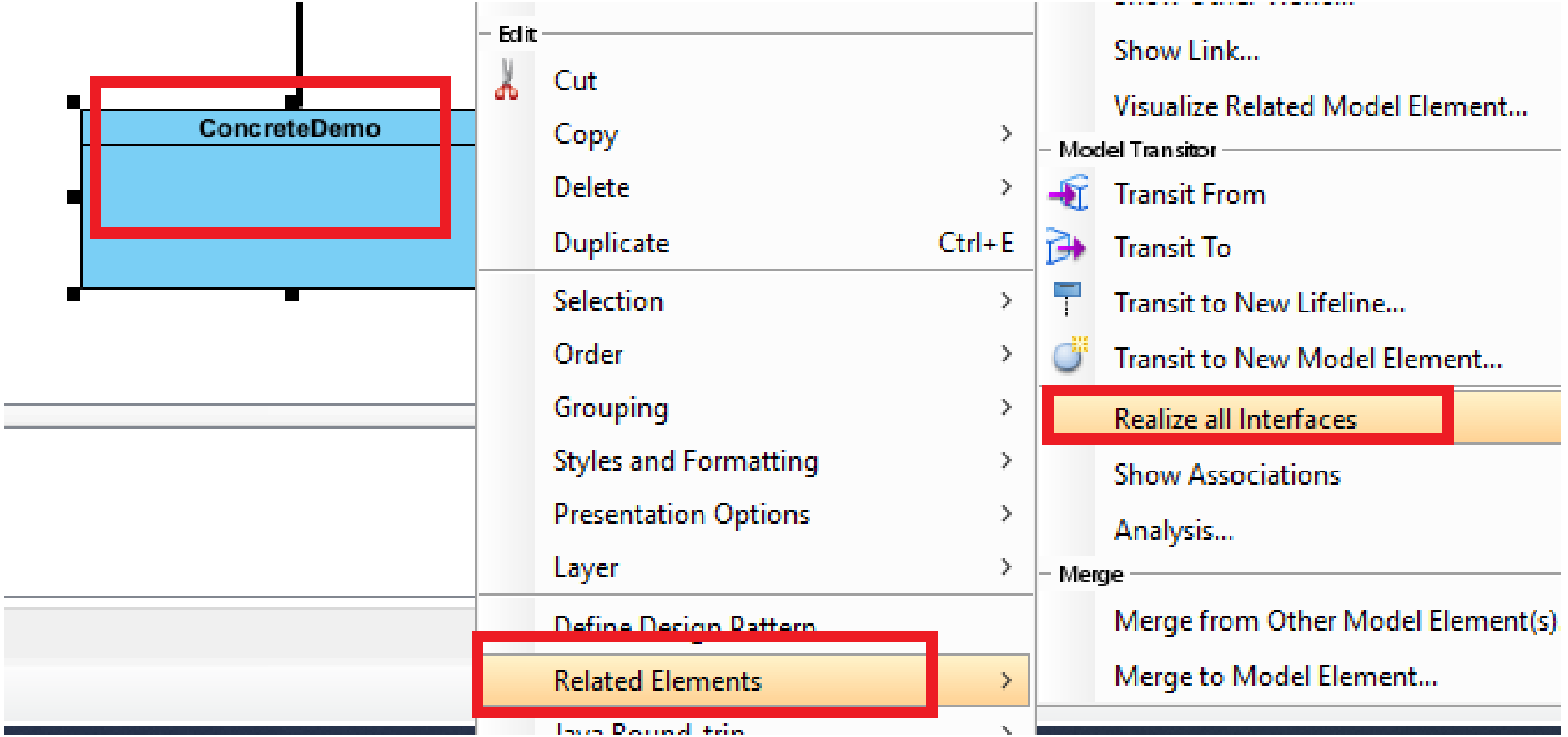


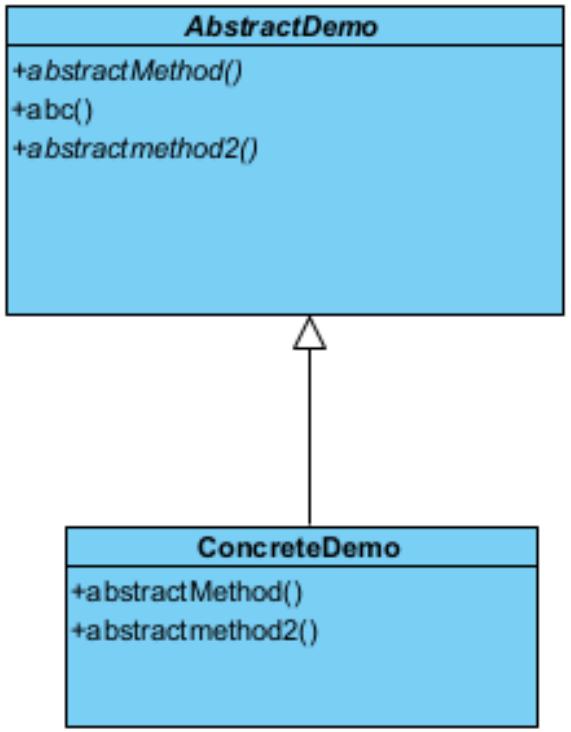


Abstract class

Abstract method



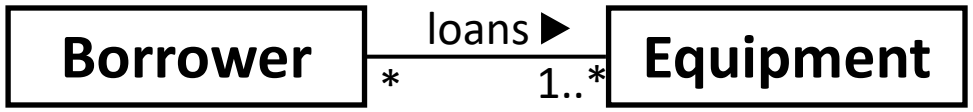

ConcreteDemo
Edit
Cut
Copy
Delete
Duplicate Ctrl+E
Selection
Order
Grouping
Styles and Formatting
Presentation Options
Layer
Related Elements
Show Link...
Visualize Related Model Element...
Model Transitor
Transit From
Transit To
Transit to New Lifeline...
Transit to New Model Element...
Realize all Interfaces
Show Associations
Analysis...
Merge
Merge from Other Model Element(s)
Merge to Model Element...

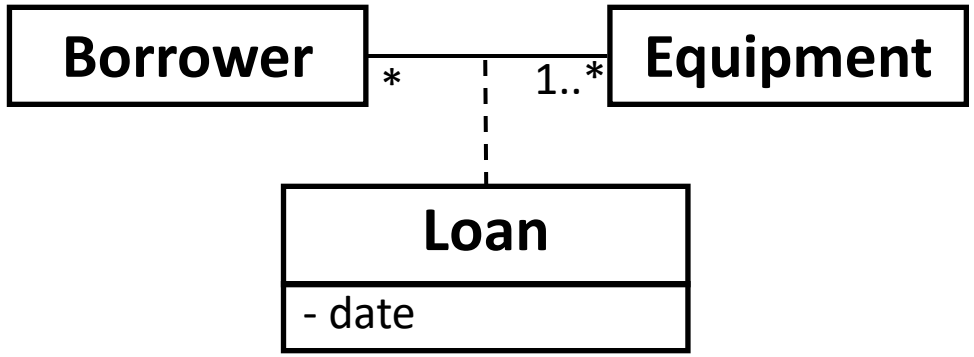



```
public class ConcreteDemo extends AbstractDemo {

	public void abstractMethod() {
		// TODO - implement
ConcreteDemo.abstractMethod
		throw new
UnsupportedOperationException();
	}

	public void abstractmethod2() {
		// TODO - implement
ConcreteDemo.abstractmethod2
		throw new
UnsupportedOperationException();
	}

}
```

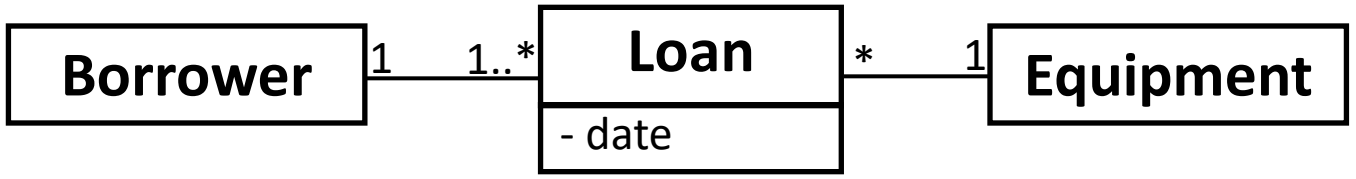
Topic 8: Advanced Concept-Association classes

Association Classes (Many to many, change to one to many + many to one)





Loan Record		
Date	Borrower	Equipment
1 Mar	Tom	Tester
11 Jul	Tom	Multimeter
1 Oct	Ann	Tester
1 Oct	Tom	Solder



An association class in object-oriented design (and UML) is a class that is used to represent the relationship between two other classes, particularly when the relationship itself has attributes or behaviors that need to be modeled.

Key Points about Association Classes:

1. **Represents a Relationship**:

   An association class is used when the relationship between two classes carries additional information or behavior that cannot be adequately described by the two classes alone. This additional information needs to be captured in a separate class.

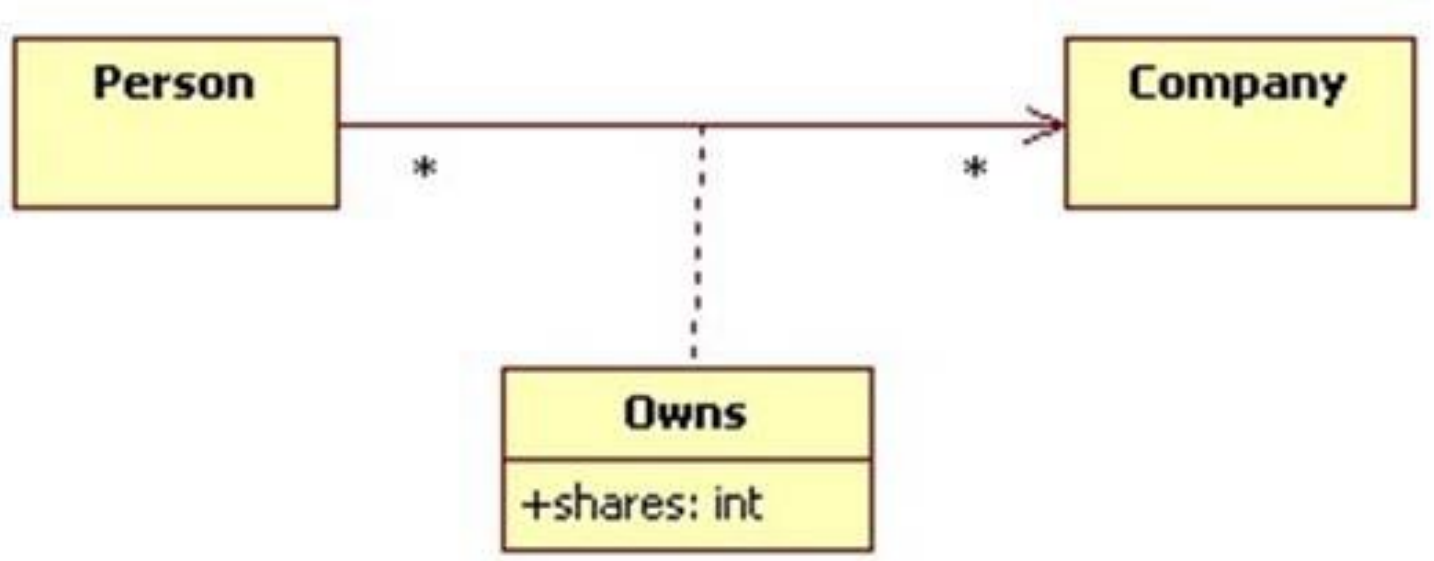
2. **Combines an Association and a Class**:

   An association class combines a regular **association** (a "has-a" relationship) with a class. It allows you to treat the association as an object in its own right, capable of holding attributes and methods.

3. **When to Use**:

   You use an association class when the relationship between two classes has its own attributes, behaviors, or constraints. It is appropriate when a relationship needs to track more than just a basic connection between two objects.

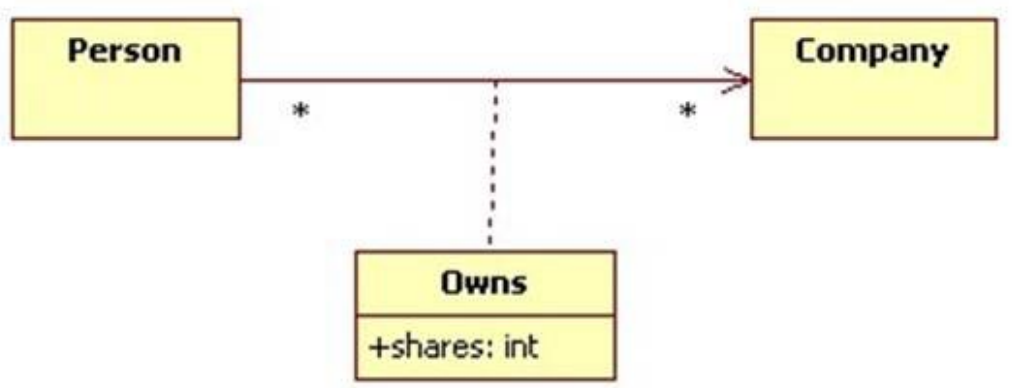
Consider the relationship "Person X owns N shares of Company Y". Where will N be stored? N is neither an attribute of Company nor Person. In cases like this we can represent links as objects. These link objects are instances of association classes:



http://www.cs.sjsu.edu/~pearce/modules/lectures/uml/class/AssociationClasses.htm

During the implementation phase an association class might be translated into Java as follows:

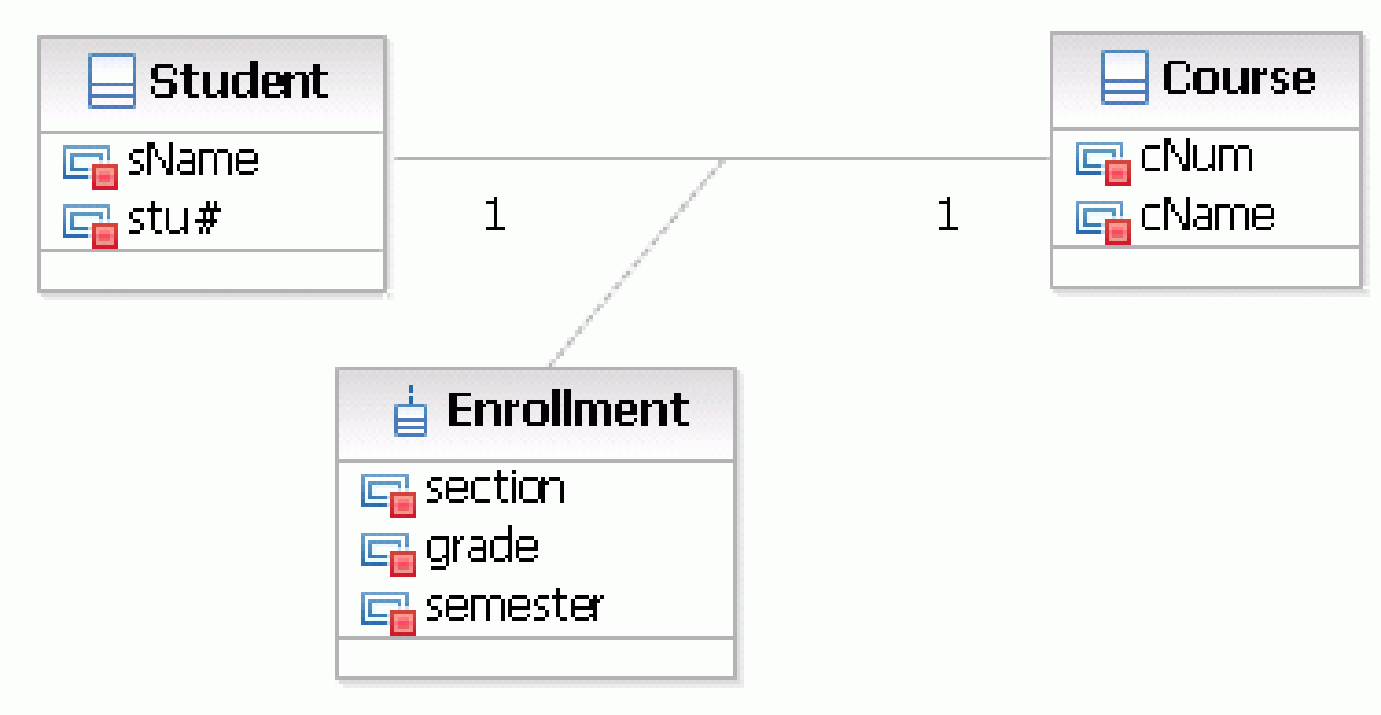
```
public class Company { ... }

public class Person {
   private List<Owns> invstments;
   public void add(Company c, int shares) {
      investments.add(new Owns(c, this, shares);
   }
   // etc.
}

class Owns {
   private Person owner;
   private Company company;
   private int shares;
   public Owns(Company c, Person p, int num) {
      comnpany = c;
      person = p;
      shares = num;
   }
   // etc.
}
```

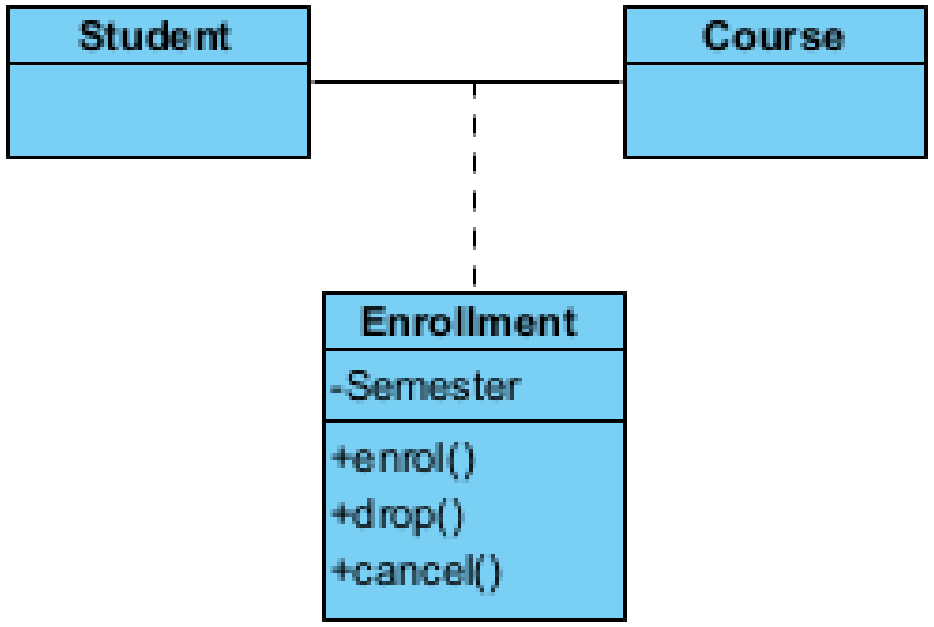


http://www.cs.sjsu.edu/~pearce/modules/lectures/uml/class/AssociationClasses

https://www.ibm.com/docs/en/rsm/7.5.0?topic=diagrams-association-classes

Consider a Student and a Course. A student can enroll in many courses, and a course can have many students. In this case, the relationship "enrollment" could be modeled as an association class because it may have attributes such as the date of enrollment or the grade for that particular course.






```
public class Student {
        private List<Enrolment>  enrolments;
        .
        .
}

public class Course {
        private List<Enrolment>  enrolments;
        .
        .
}

public class Enrolment {
        private Person Student;
        private Company Course;
        .
        .
}
```

1. **Primary Role of Association Class**:
    1. The **association class** is primarily designed to encapsulate attributes or behaviors specific to the relationship between two classes. For instance, in the **Student** and **Course** example, the **Enrollment** association class captures details about that relationship (e.g., enrollment date, grade).

2. **Access from Other Classes**:
    1. Although the **association class** is mainly tied to the relationship between two classes, it is still a regular class and can have methods, attributes, and associations like any other class. Therefore, it **can** be referenced, used, or associated with other classes in the system.
    2. For example, another class could interact with the **Enrollment** class to access information about the student's enrollment details.

- An **association class** can be used by other classes beyond the two it connects, as it is a regular class that can be associated or interacted with like any other class in the system.
- For example, other classes, like a **Report** or **Billing** class, can use the association class to retrieve or manipulate data relevant to the relationship (such as grades or fees). However, the decision to do so depends on the design and the context of the application.

- **Use Cases of Association Classes with Other Classes:**

1.Example 1: A Report Class Accessing Enrollment Data:

1. Suppose there is a **Report** class that generates various reports about student performance. The **Report** class could directly use the **Enrollment** class to access details about student grades for specific courses.
    1. The **Report** class might query the **Enrollment** class to retrieve all the grades of a particular student or all students enrolled in a course.

1. **Example 2: A Financial System Using Enrollment Class**:

   If the **Enrollment** class also contains financial information, such as tuition fees for the course, a **Payment** or **Billing** class could interact with the **Enrollment** class to process payments based on the course fees or generate invoices for the student.

- **Key Points to Consider:**

- **Flexible Use**: While the **association class** serves to represent the relationship between two specific classes, other classes can still use it for its attributes and behaviors, as long as the system's design makes this logical.

- **Design Decision**: Whether an association class should be used by other classes is a design decision that depends on the relationships and interactions within the system. If the information captured by the association class is relevant to other parts of the system, it makes sense to allow other classes to access or use it.

Topic 9: ArrayList

```
public class VendingMachine
{
        Drink[] drinks ;  // object composition - relevant
        double drinkCost = 0.0;
        int drinkSelection = 0;

    public VendingMachine() {
            drinks = new Drink[3];
```

Java ArrayList

https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html

The `ArrayList` class is a resizable array, which can be found in the `java.util` package.

The difference between a built-in array and an `ArrayList` in Java, is that the size of an array cannot be modified (if you want to add or remove elements to/from an array, you have to create a new one). While elements can be added and removed from an `ArrayList` whenever you want. The syntax is also slightly different:

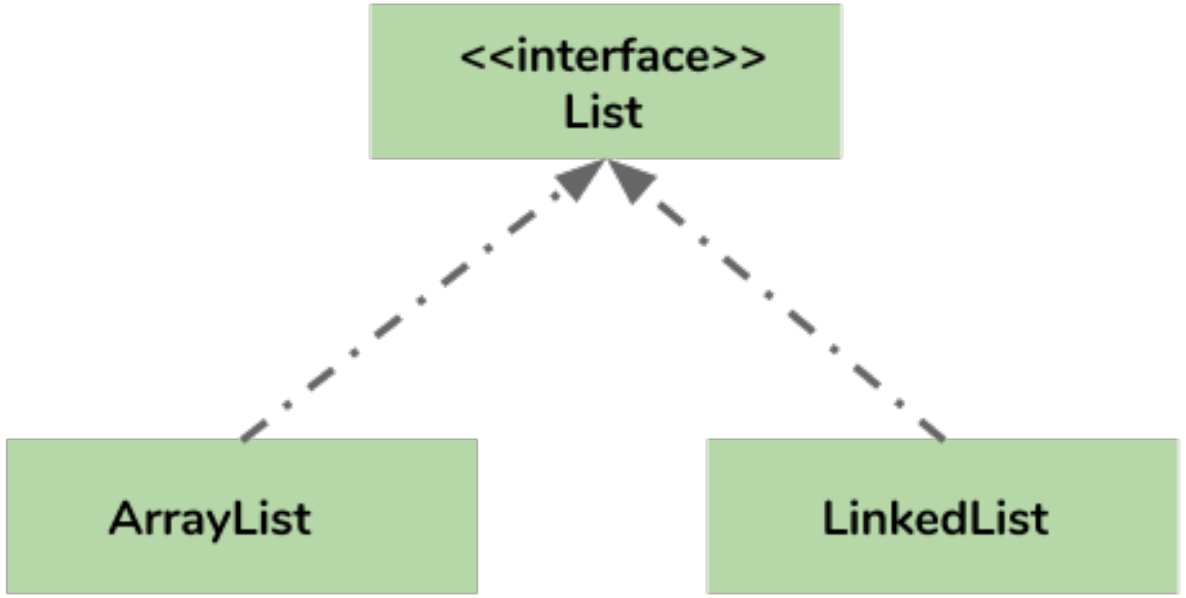
Example

Create an `ArrayList` object called **cars** that will store strings:

```
import java.util.ArrayList; // import the ArrayList class

ArrayList<String> cars = new ArrayList<String>(); // Create an ArrayList object
```

https://www.w3schools.com/java/java_arraylist.asp

- Arraylist class implements List interface and it is based on an Array data structure.
- Most of the developers choose Arraylist over Array as it's a very good alternative of traditional java arrays.
- ArrayList is a resizable-array implementation of the List interface.



Add Items

The `ArrayList` class has many useful methods. For example, to add elements to the `ArrayList`, use the `add()` method:

Example

```
import java.util.ArrayList;

public class Main {
  public static void main(String[] args) {
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Volvo");
    cars.add("BMW");
    cars.add("Ford");
    cars.add("Mazda");
    System.out.println(cars);
  }
}
```

https://www.w3schools.com/java/java_arraylist.asp

slido

Which of the following is the correct way to get the first value in a list called cars?

ⓘ Start presenting to display the poll results on this slide.

Access an Item

To access an element in the `ArrayList`, use the `get()` method and refer to the index number:

Example

```
cars.get(0);
```

Change an Item

To modify an element, use the `set()` method and refer to the index number:

Example

```
cars.set(0, "Opel");
```

Remove an Item

To remove an element, use the `remove()` method and refer to the index number:

Example

```
cars.remove(0);
```

To remove all the elements in the `ArrayList`, use the `clear()` method:

Example

```
cars.clear();
```

slido

What will print when the following code executes?

```
List<String>  list1 = new ArrayList<String>();
list1.add("Anaya");
list1.add("Layla");
list1.add("Sharrie");
list1.set(0, "Destini");
list1.add(0, "Sarah");
System.out.println(list1);
```

https://runestone.academy/ns/books/published/apcsareview/ListBasics/listEasyMC.html

ⓘ Start presenting to display the poll results on this slide.

```java
import java.util.ArrayList;

class Vendingmachine {
private ArrayList<Drink> drinks;

public Vendingmachine(ArrayList<Drink> drinks) {
this.drinks=drinks;
}

public void DisplayDrink(){
// Printing elements one by one
        for (int i = 0; i < drinks.size(); i++)
            System.out.println(drinks.get(i).getName());
}
}
```

```java
class Drink {
private String name;
private double cost;

public Drink(String n,
double c) {
name = n ;
cost = c;
}
public String getName() {
return name;}
public double getCost() {
return cost;}
}
```

```java
public class VMApp {
public static void main(String[] args) {
ArrayList<Drink> drinks = new ArrayList<Drink>();
drinks.add(new Drink("Beer" ,3.00));
drinks.add(new Drink("Coke" ,1.00));
Vendingmachine vm = new Vendingmachine(drinks) ;
vm.DisplayDrink();
}}
```

→ Beer
Coke

Reference

- Google images